

基于多因素风险量化的 NIPT 时点优化与异常判定分析

摘要

本文针对 NIPT 的时点选择与胎儿的异常判定等问题，使用 LASSO 回归，随机森林等方法进行特征的筛选，再针对各问题分别进行多风险建模求解，最后解决 BMI 分组策略，NIPT 时点选择与胎儿异常判定等问题。

针对问题一，主要分析胎儿 Y 染色体浓度与孕妇孕周、BMI 等指标的相关特性，并检验了显著性。本文首先针对多维度变量数据，进行缺失值处理、异常值检测和标准化。然后采用 **LASSO 回归** 筛选关键变量，构建 **多元线性回归模型** 和广义可加模型 (**GAM**)。通过交叉验证计算 MSE、 R^2 比较模型性能，结果表明，GAM 性能优于多元线性回归模型。并且两种模型整体高度显著 (p 值 <0.001)。同时 X 染色体浓度对 Y 染色体浓度预测贡献最大 (回归系数=0.3882)，呈高度显著正向影响。检测抽血次数 (回归系数=0.0166) 和检测孕周 (回归系数=-0.0031) 影响力次之。

针对问题二，基于 BMI 与 Y 染色体浓度达标孕周的关联性，首先构建 **决策树** 模型进行 BMI 分组。通过分析 BMI 与 Y 染色体浓度首次达到 4% 的孕周关系，采用决策树算法递归划分 BMI 区间，以最小化潜在风险为目标，确定各 BMI 区间对应的最佳 NIPT 检测时点。分析结果显示，BMI 可划分为 5 个区间：对 [20.70, 30.48]、(30.48, 34.50]、(34.52, 35.29]、(35.32, 36.89] 和 (36.93, 45.71]，应最佳 NIPT 检测时点分别为 14.00 周、13.46 周、16.02 周、14.81 周和 19.77 周。然后基于男胎检测数据，计算各组过早/过晚检测概率加权得到总风险，给出建议：高 BMI 组 (>34.5) 达标孕周显著延迟，过早检测风险高，建议推迟检测。最后通过灵敏度分析，检测误差表明，浓度阈值在 $4\% \pm 0.2\%$ 波动时，推荐检测孕周变化小于 0.5 周，进一步论证了该模型稳健性良好。

针对问题三，需要在问题二的基础上加上额外的限定条件。并通过 **随机森林** 进行特征重要性排序。然后构建 **广义可加混合模型**，结合多目标优化与误差分析方法，捕捉非线性关系，通过加权目标函数 (最小化风险与最大化达标比例) 实现多目标优化。结果表明，模型将 BMI 划分为 5 个区间：[27.9, 39.2]、[28.1, 35.0]、[26.6, 40.1]、[28.3, 32.7]、[20.7, 46.9]，对应的最佳 NIPT 时点分别为 12.0 周、14.1 周、15.2 周、14.3 周、16.6 周。

针对问题四，针对女胎异常判定问题，本文采用两层结构的 **分层级联判定模型**：基于逻辑回归的 **筛查模型** 和基于 SVM 的 **专家模型**。通过随机森林筛选特征，采用 SMOTE 处理类别不平衡，对异常样本进行分析。结果表明，筛查模型成功识别所有异常样本，T21 异常识别率达 100%，T18 异常识别率达 86%。特征重要性分析显示，13 号染色体 GC 含量、18 号染色体 GC 含量及 21 号染色体 Z 值是判定异常的关键指标。

关键词：多元线性回归 LASSO 回归 GAM 决策树 随机森林 SVM 专家模型

一. 问题重述

NIPT 是采集母体血液、检测胎儿游离 DNA 片段以分析胎儿染色体是否异常的产前检测技术，目的是早期确定胎儿健康状况。临床中，畸形胎儿主要为唐氏综合征、爱德华氏综合征和帕陶氏综合征，分别由胎儿 21 号、18 号、13 号染色体浓度异常导致。NIPT 准确性主要看胎儿性染色体浓度，孕妇孕期 10 至 25 周可检测该浓度：男胎 Y 染色体浓度 $\geq 4\%$ 、女胎 X 染色体浓度无异常时，结果基本准确，否则难以保证准确性。且胎儿异常发现时间关乎风险，12 周内发现风险低，13-27 周风险高，28 周后风险极高。

实践显示，男胎 Y 染色体浓度^[7]与孕妇孕周、BMI 紧密相关，临床常按 BMI 分组（如 [20, 28）、[28, 32) 等）确定检测时点。但孕妇年龄、BMI、孕情存在个体差异，简单分组和统一时点会影响 NIPT 准确性，故需依据 BMI 合理分组，确定各组最佳检测时点，以减少因胎儿不健康导致治疗窗口期缩短的潜在风险。



图 1 无创产前检测与孕期（来源网络）

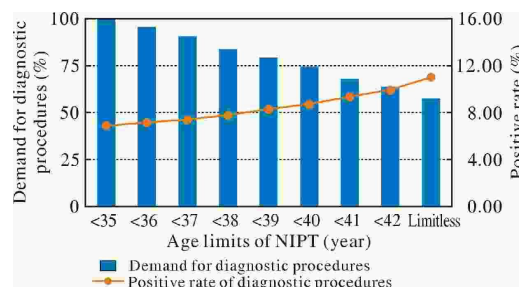


图 2 无创产前检测的年龄限制（来源网络）

基于以上背景和附件，可以建立数学模型来解决以下问题：

1. 分析胎儿 Y 染色体浓度与孕妇孕周、BMI 等指标的相关性，构建关系模型并检验显著性；
2. 依据 BMI 对男胎孕妇合理分组，明确每组 BMI 区间与最佳 NIPT 时点以降低潜在风险，并分析检测误差的影响（BMI 是 Y 染色体浓度达标时间主因）；
3. 综合身高、体重等因素及检测误差、Y 染色体浓度达标比例，结合 BMI 对男胎孕妇分组，确定最佳 NIPT^[8]时点以降低风险，并分析检测误差影响；
4. 结合女胎孕妇相关染色体非整倍体结果，及 X 染色体等的 Z 值、GC 含量等因素，给出女胎异常判定方法。

二. 问题分析

2.1 问题一的分析

问题一要求分析胎儿 Y 染色体浓度与孕妇孕周、BMI 等指标的相关特性，建立关系模型并检验显著性。题目明确指出男胎 Y 染色体浓度与孕妇孕周数及 BMI 紧密相关，且 Y 染色体浓度是判断 NIPT 结果准确性的关键指标（ $\geq 4\%$ 为达标）。数据中包含多维度变量：孕周、BMI、年龄、身高、体重、X 染色体浓度、Z 值、GC 含量等，其中孕周与 BMI 是核心自变量，而 Y 染色体浓度为因变量。考虑到变量间可能存在多重共线性，需进行数据预处理，包括缺失值处理、异常值检测、数据标准化及变量筛选。初步探索分析显示 Y 染色体浓度与 X 染色体浓度呈强正相关，与 BMI、体重呈弱负相关，表明变量间存在复杂关系。

为确定最优模型，需权衡线性与非线性关系。首先采用 LASSO 回归^[5]进行变量筛选，确定对 Y 染色体浓度影响显著的变量。随后，构建多元线性回归模型^[1]评估变量的线性影响，同时建立广义可加模型（GAM）^[2]捕捉潜在的非线性关系。通过交叉验证计算 MSE、 R^2 。显著性检验将采用 F 检验评估整体模型显著性，t 检验评估各变量系数的显著性。通过比较不同模型的拟合效果与变量重要性，确定 Y 染色体浓度与孕周、BMI 等指标的具体关系特性，为后续问题提供数据支撑。

2.2 问题二的分析

问题二要求对男胎孕妇的 BMI 进行合理分组，确定每组 BMI 区间和最佳 NIPT 时点，以最小化潜在风险，并分析检测误差影响。分析发现，BMI 与 Y 染色体浓度达标孕周存在非线性关系，且不同 BMI 组的达标孕周分布差异显著。于是本文首先采用决策树模型进行 BMI 分组，以最小化潜在风险为目标函数。决策树递归划分 BMI 区间，通过最小化各区间内达标孕周的方差和风险加权值，确定最优分组。接着计算各组过早/过晚检测概率，构建风险量化模型。然后，为分析检测误差影响，设置浓度阈值波动范围（如 $4\% \pm 0.2\%$ ），观察推荐检测孕周的变化，进而评估模型稳健性。由于决策树模型能自动处理变量间复杂关系，避免主观分组导致的误差，同时考虑了两个关键因素：各 BMI 组内 Y 染色体浓度首次达到 4% 的孕周分布；不同检测时点导致的过早/过晚检测风险，从而充分证明分组结果的合理性。

2.3 问题三的分析

问题三在问题二基础上，进一步考虑身高、体重、年龄等多因素对 Y 染色体浓度达标时间的影响，还需处理多维因素与纵向数据特性。因此，本文采用能捕捉个体差异的广义可加混合模型(GAMM)处理多因素影响。其中固定效应通过平滑函数捕捉非线性关系，随机效应则建模个体间差异。此涉及多目标优化：最小化潜在风险和最大化达标比例。为此，构建加权目标函数，将风险与达标比例转化为统一优化指标。接着采用协调迭代法求解，直至目标函数收敛。同时，考虑检测误差对结果的影响，通过灵敏度分析评估模型稳健性。

2.4 问题四的分析

问题四要求以女胎孕妇的 21、18 和 13 号染色体非整倍体为判定结果，综合考虑多种因素，建立女胎异常判定方法。本文构建分层级联判定模型：第一层采用逻辑回归建立筛查模型，通过调整概率阈值实现高召回率，确保无异常样本被遗漏；第二层采用“一对多”策略构建 SVM 专家模型，将三分类问题分解为三个二分类子问题，分别对 T21、T18、T13 异常进行精准识别；最后，通过特征重要性分析确定关键指标，并对模型进行可解释性分析确定其合理性

三. 模型假设

1. 假设附件所提供的的所有数据，包括孕妇的个人信息、各项生理及测序指标、以及最终的异常判定结果（AB 列），均是真实、准确的；
2. 假设在数据采集期间及未来预测期内，不存在未被数据反映的、对 NIPT 检测结果和临床决策产生颠覆性影响的重大外部事件（如技术突变、重大政策调整、新的流行病爆发等）；
3. 假设 Y 染色体浓度达到或超过 4% 作为 NIPT 结果基本准确的判断标准，在整个研究期间是固定且有效的；

4. 假设 BMI 是影响男胎 Y 染色体浓度最早达标时间的主要因素，在进行分组决策时，可以优先甚至单独考虑该因素；
5. 假设复杂的多分类问题可以被有效地分解为一个“筛查-分型”的两阶段任务，并且这种分解不会损失关键的分类信息。

四. 符号说明

表 1 本文中使用的符号

符号	意义
x_i	附件中各自变量
Y_1	多元线性回归模型
Y_2	广义可加模型
$t_{\min}^{(j)}$	第 j 个样本达标时间
t^*	达标时间置信上界
R_i	总风险函数
$y_i(t)$	Y 染色体浓度预测函数
F	优化目标函数
C_k	异常类别集合
h_k	分类器输出概率

五. 问题一的模型建立与求解

5.1 思路分析

首先，通过数据探索与相关性分析，从直观和定量的角度揭示变量间的关系模式；然后，对比建立两种不同复杂度的回归模型——经典的多元线性回归模型作为基准，以及更灵活的广义可加模型（GAM）来探索非线性关系；最后，通过模型检验来评估所建模型的科学性和有效性。

5.2 数据处理与探索

附件内容中数据格式多样，为了探究多变量对 Y 染色体浓度的影响，和数据之间是否存在关联，需要对数据处理挖掘。

5.2.1 数据预处理

男胎样本包括：总样本量 1082 例男胎检测数据，Y 染色体浓度数据完整率 100%，时间跨度覆盖完整孕期检测窗口。对于需要探究的变量我们可以大致进行分类如下：

- 连续变量：年龄、身高、体重，孕妇 BMI、检测孕周等；

- 二分类变量：胎儿健康状况（是/否）；
 - 多分类变量：妊娠方式（自然受孕/IUI/IVF）；
- 针对多类型变量，我们采用以下方法进行数据处理：

● 完整性检查与格式标准化

在数据处理中，对于 Y 染色体缺失值，删除其记录。针对孕周的复杂格式，如 12w+3,12+,12w+ 进行智能格式分析，映射为 12.43 周，12.0 周，12.0 周。分类类型变量：妊娠方式（自然受孕/IUI/IVF）对应 0，1，2。胎儿健康状况（是/否）对应 1，0。

● 异常值检测

对于附件中的数据，采用四分位距（IQR）方法进行异常值检测。针对检测出的异常极端数据，例如 Y 染色体 Z 值最大值为 7.000856057，进行移除。通过处理，得到 925 例优质数据，提高了数据质量，使得统计功效和结果可靠性提升。

5.2.2 数据探索

对 Y 染色体浓度数据进行统计观察，其均值为 7.79%，最大值 17.23%，最小值 1.16%，大于 4% 的达标率为 89.8%。整理并绘制结果如下图：

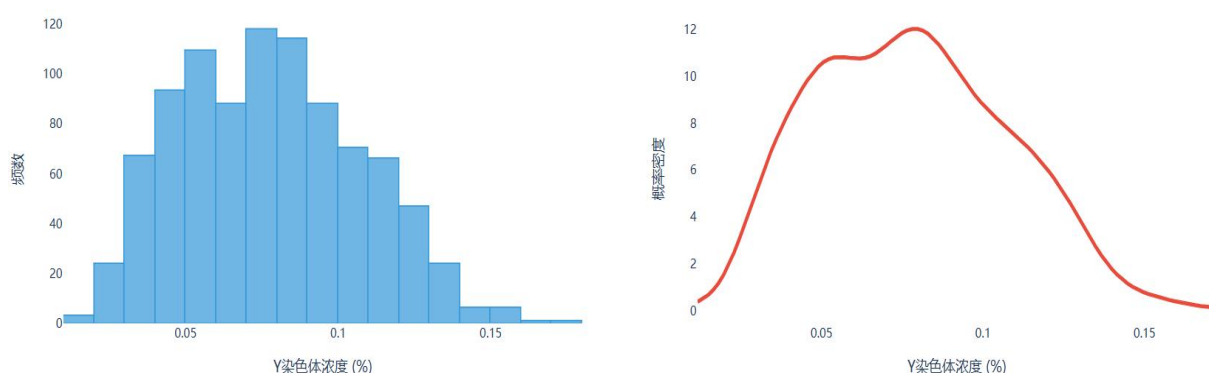


图 3 Y 染色体浓度分布直方图（左）密度图（右）

观察左侧频数分布直方图，分布形态为略右偏的近正态分布，Y 染色体浓度 >4% 的占比为 89.8%，多数数据高于阈值 4%，说明进行检测 NIPT 结果大部分基本准确。右侧密度曲线图，消除了直方图的锯齿效应，显示真实分布形态。右尾略长，存在少数高浓度个体，无明显断点，良好的连续分布特征表明生物学过程连续性。

同时，为了进一步探查变量与 Y 染色体浓度的相关性，我们进行了初步的连续数据分析，并将部分数据散点图绘制如下：

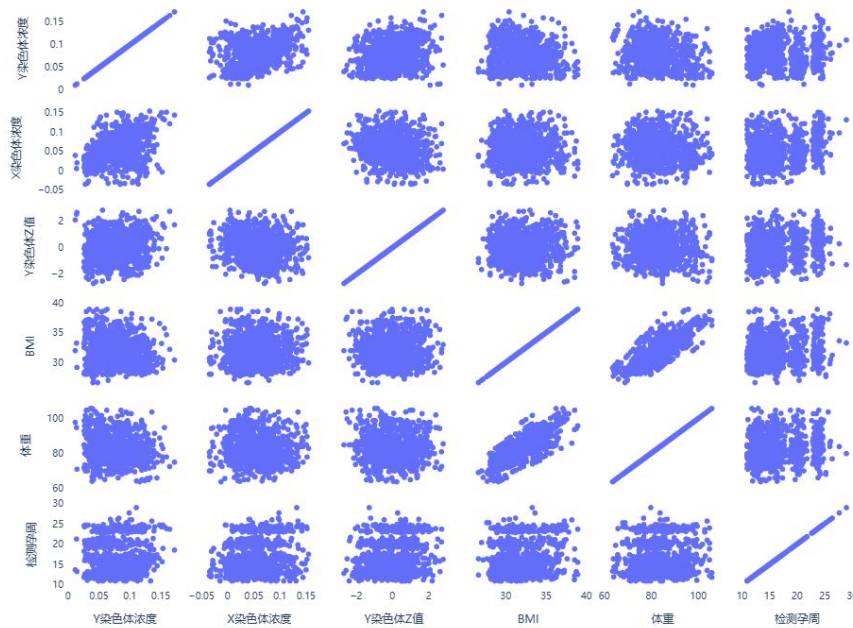


图 4 连续变量散点矩阵图

分析矩阵图可知，在众多变量中：

- Y 染色体浓度与 X 染色体浓度呈现强正线性关联 ($r=0.420$)，散点呈明显上升趋势，有少数离群点，但整体趋势明确。这可能与 X、Y 染色体来自同一胎儿细胞，拥有共同释放机制有关；
- Y 染色体浓度与 BMI 和体重呈现弱负相关 (BMI: $r=-0.123$ ，体重: $r=-0.120$)，散点呈轻微下降趋势。高 BMI 或高体重者 Y 染色体浓度偏低。存在个体差异，但趋势明确。可能原因是体重增加导致血容量增加，进而使得 DNA 相对稀释。
- 整体上来说，存在许多变量与 Y 染色体浓度并不存在明显相关性。

5.3 基于 LASSO 回归模型的自变量选择

首先对于题目提到的孕周数和 BMI，我们直接作为核心变量保存。进一步，通过对数据的初步分析可知，Y 染色体浓度受多个变量影响，并且变量的相关性强度差异较大，这使得变量的筛选尤为重要。为此，我们构建 LASSO 回归模型进行变量的自动选取。LASSO 回归模型是针对多元线性回归模型的改进，它通过最小化残差平方和最小化 (残差平方和 + λ * 系数绝对值之和)，可以将不重要的特征系数压缩为 0，实现自动特征选择。

5.3.1 LASSO 模型建立

先对所有变量进行标准化处理，使其均值为 0，方差为 1：

$$x'_{ij} = \frac{x_{ij} - x_j}{s_j} \quad (1)$$

其中 x_j 表示第 j 个样本的样本均值， s_j 表示样本标准差。

然后对因变量 y 进行中心化处理：

$$y'_i = y_i - \bar{y} \quad (2)$$

其中 $\bar{y}$ 表示 Y 染色体浓度的样本均值。

以此建立 Lasso 回归模型选择自变量：

$$y = \beta_1 x'_1 + \beta_2 x'_2 + \beta_3 x'_3 + \dots + \beta_{14} x'_{14} + \varepsilon \quad (3)$$

进一步，通过 Lasso 估计，最小化下面目标函数：

$$\hat{\beta} = \arg \min_{\beta} \left\{ \frac{1}{2n} \sum_{i=1}^n (y'_i + \sum_{j=1}^{14} \beta_j x'_{ij} + \lambda \sum_{j=1}^{14} |\beta_j|) \right\} \quad (4)$$

5.3.2 自变量选择

在 LASSO 回归模型中，正则化惩罚强度参数 λ 代表惩罚力度，我们使用 python 中 LassoCV 这个强大的工具：CV 代表交叉验证，它会自动用数据测试每一个 λ 值。通过不同的 λ 值来锁定哪些变量的回归系数能够承受较大的惩罚力度而不被压缩为 0，那么该变量就是最重要的特征。具体 LASSO 回归路径图如下：

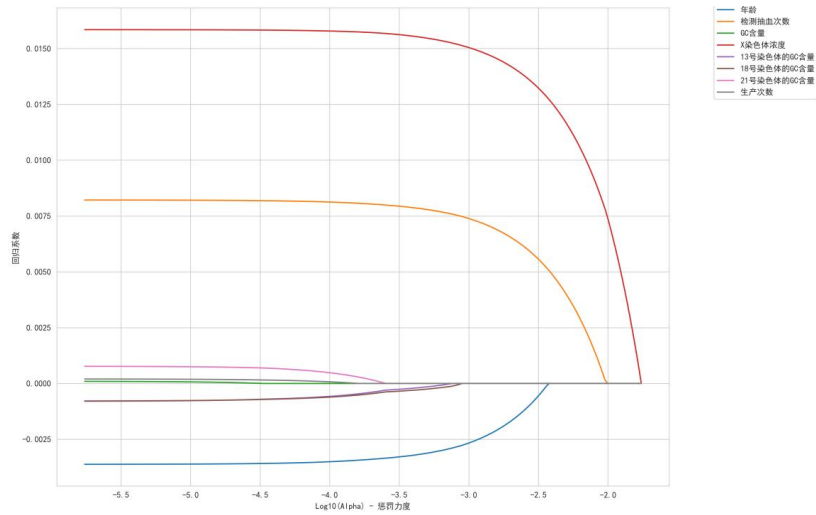


图5 LASSO 回归路径图

如上图所示，横轴代表惩罚力度大小，纵轴代表回归系数。其中 X 染色体浓度能够承受柱最大的惩罚力度，是最重要的变量，其次是抽血检测次数和年龄。而生产次数和 GC 含量的回归系数则很早就被压缩为 0，相对并不重要。

通过 LASSO 回归^[10]进行变量的筛选，我们决定使用筛选出来的 X 染色体浓度，抽血检测次数和年龄。再加上题目提及的核心变量孕周数和 BMI 作为后续模型建立的指导。

5.4 模型建立

5.4.1 多元线性回归模型

选取孕周数和 BMI，X 染色体浓度，抽血检测次数和年龄作为输入构建多元线性回归模型：

$$Y_1 = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_5 X_5 + \varepsilon \quad (5)$$

其中 $x_1, x_2, \dots, x_5$ 分别对应 5 个自变量, $\beta_0, \beta_1, \beta_2, \dots, \beta_5$ 表示回归系数, ε 表示误差项。

使用最小二乘法 OLS 估计参数:

$$\hat{\beta} = (X^T X)^{-1} X^T Y \quad (6)$$

其中 X 表示包含常数项以及所有输入变量的矩阵, Y 表示响应变量的向量。

5.4.2 广义可加模型建立

建立广义可加模型 (GAM) :

$$Y_2 = f_1(X_1) + f_2(X_2) + \dots + f_5(X_5) + \varepsilon \quad (7)$$

其中 $f_i()$, $i=1, 2, \dots, 5$ 为平滑函数, 表示为:

$$f_j(X_j) = \sum_{k=1}^{K_j} \beta_{jk} B_{jk}(X_j) \quad (8)$$

其中 K_j 表示基函数数量, β_{jk} 表示待估参数, B_{jk} 表示第 k 个基函数, 其余变量含义与回归模型一致。

通过惩罚似然函数估计参数:

$$\hat{\beta} = \arg \min_{\beta} \{ \|Y - X\beta\|^2 + \sum_{j=1}^p (\lambda_j \beta^T S_j \beta) \} \quad (9)$$

其中 λ_j 表示平滑参数, S_j 表示惩罚矩阵。

5.5 模型结果与显著性检验

5.5.1 模型结果分析

通过两种不同的模型对数据分别进行拟合, 得到结果如下图所示:

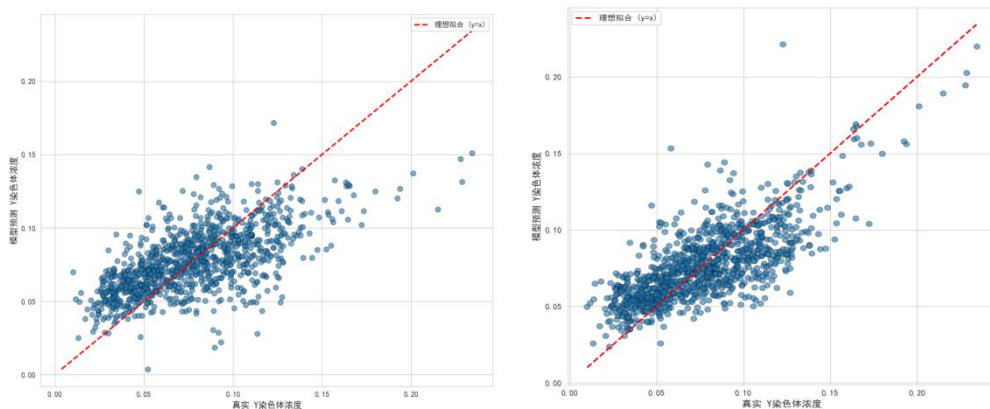


图 6 多元线性回归模型 (左) 和广义可加模型 (右) 拟合效果图

两个模型对 Y 染色体浓度预测都有相对准确的效果, 但是相比之下多元线性回归模型偏差更大, 尤其是在高浓度区域, 显示出较大的预测误差。而在 GAM 模型中, 预测点更佳紧密地聚集在完美预测线附近, 特别是低浓度和高浓度区域显示出较小的预测误差。同时我们能得到两个模型性能如下:

表 2 模型性能评价指标对比

模型	评价指标		
	R^2	均方误差 (MSE)	平均绝对误差(MAE)
多元线性回归模型	0.4208	0.000650	0.020089
GAM	0.5555	0.000499	0.17669

如上表所示,多元线性回归的 $R^2=0.428$ 而 GAM 模型 $R^2=0.5555$, GAM 模型的解释能力更强。同时 GAM 在均方误差与平均绝对误差上也优于多元线性回归模型,说明 GAM 模型对 Y 染色体浓度预测有着更强的能力。这也才能够侧面看出数据呈现的是更为复杂的非线性关系。

5.5.2 显著性检验

(1) F 检验:

$$F = \frac{MSR}{MSE} = \frac{SSR / P}{SSE / (n - p - 1)} \quad (10)$$

其中 $SSR = \sum (\hat{Y}_i - \bar{Y})^2$ 表示回归平方和, $SSE = \sum (Y_i - \hat{Y}_i)^2$ 表示残差平方和, n 表示样本量, p 表示预测变量数。原假设 $H_0: \beta_1 = \beta_2 = \dots = \beta_p = 0$ 。F 值大于临界值或者 $p < 0.05$: 则模型整体显著。

(2) t 检验:

对于每个系数 β_j 都有:

$$t_j = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)} \quad (11)$$

其中 $SE(\hat{\beta}_j)$ 表示系数标准误差,服从自由度为 n-p-1 的 t 分布。

并最终得到 F 检验结果为: F 统计量: 156.38; P 值: 6.2430e-125。得出结论: P 值远小于 0.05, 因此拒绝原假设 (所有系数均为 0), 模型整体高度显著。

同时 t 检验结果为:

表 3 多元线性回归模型 t 检验表

变量	评价指标				
	系数	标准误差	t 统计量	p 值	95%置信区间
常量	0.1480	0.011	13.658	0.000	[0.127,0.169]

变量	评价指标				
	系数	标准误差	t 统计量	p 值	95%置信区间
检测孕周	-0.0031	0.000	-10.010	0.000	[-0.004,-0.002]
孕妇 BMI	-0.0017	0.000	-6.342	0.000	[-0.002,-0.001]
年龄	-0.0010	0.000	-4.537	0.000	[-0.001,-0.001]
检测抽血次数	0.0166	0.001	14.959	0.000	[0.014,0.019]
X 染色体浓度	0.3882	0.019	20.264	0.000	[0.351,0.426]

表中列出的所有变量，其 p 值均为 0.000，均小于常用的显著性水平（0.05），说明所有变量均显著。这意味着所有这些变量都是预测因变量的显著因素。基于回归系数，我们可以看出检测抽血次数 和 X 染色体浓度为正向影响（系数为正）它们的增加会导致因变量增加。孕妇 BMI 和 年龄负向影响（系数为负），它们的增加会导致因变量减少。最强影响：X 染色体浓度（回归系数=0.3882）。最弱影响：年龄（回归系数=-0.0010）。

对于 GAM，其整体显著性通过各项的显著性来体现，若各项都显著，那么整体也显著。各变量的显著性检验使用平滑项检验，并得到结果：

表 4 GAM 平滑项检验表

变量	评价指标			
	平滑参数	秩	有效自由度	p 值
检测孕周	0.6	20	13.6	1.11e-16
孕妇 BMI	0.6	20	10.2	3.93e-12
年龄	0.6	20	12.0	5.89e-09
检测抽血次数	0.6	20	3.7	1.11e-16
X 染色体浓度	0.6	20	11.2	1.11e-16

首先，表中所有五个平滑项 $s(0)$ 到 $s(4)$ 的 p 值都远小于 0.001。这意味着我们强烈拒绝原假设，这些变量的显著性都相当高。其次有效自由度能够衡量平滑函数的实际复杂度。

有效自由度=1：意味着该变量的关系几乎是一条直线。

有效自由度>1：意味着存在非线性关系。EDoF 越大，曲线越弯曲、越复杂。

例如，检测抽血次数的有效自由度为 3.7，非线性程度较低；而检测孕周为 13.6，表明其与因变量的关系非常复杂，非线性很强。绘制 GAM 的平滑效应图如下：

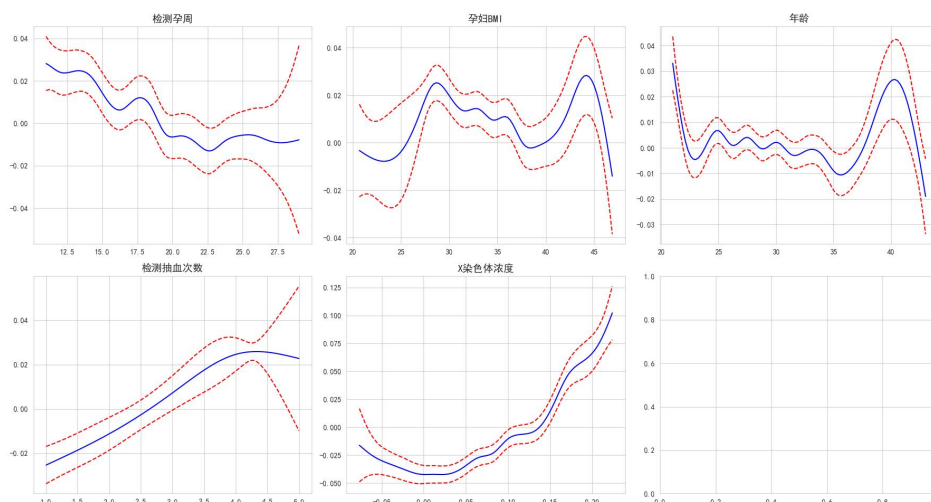


图 7 GAM 模型各变量的平滑效应图

图中蓝色实线是模型学到的核心关系曲线。它的形状揭示了变量影响 Y 浓度的具体方式（直线或曲线）。红色虚线表示 95% 的置信区间。

从图表中可以得出具体结论：

- 对于孕妇 BMI：这条曲线在 BMI 值较低时比较平缓，甚在 BMI=25 时开始急速上升，但一旦 BMI 超过 44，曲线会急剧下降。而在此区间内部则变化平缓。这说明：对于体重正常的孕妇，BMI 影响不大；但对于肥胖孕妇，BMI 的增加会显著地、加速地降低 Y 染色体浓度；
- 对于检测孕周：曲线呈现一个先下降后平缓的趋势。在孕早期（12-20 周），Y 浓度随孕周增加而明显下降；但在孕中期以后（20 周后），这种下降趋势就基本停止了。这说明孕周的影响主要集中在特定的时间窗口内；
- 对于年龄：曲线整体呈下降趋势，但在 35 岁之后，下降的坡度会更陡，说明高龄对 Y 浓度的负面影响更大；
- 对于 X 染色体浓度和检测抽血次数：这两条曲线近似于直线，稳定地从左下角向右上角延伸，表明它们的影响是持续、正向的。

至此，本文使用多元线性回归与 GAM 分别从线性与非线性两中可能关系预测了 Y 染色体浓度，并且都取得了高显著性的结果。

六. 问题二的模型建立与求解

6.1 思路分析

本题对于男胎孕妇 BMI 分组及最佳 NIPT 时点问题的解题思路由以下部分组成：首先基于 BMI 与 Y 染色体浓度的非线性关系，采用决策树算法划分 BMI 区间，以最小化过早/过晚检测风险为目标函数。通过计算各组达标孕周分布，构建风险量化模型^[4]，确定最优分组边界。结合检测误差分析，评估模型稳健性。最终输出 BMI 分组区间及对应的最佳检测时点，为临床提供个体化建议，降低误判风险并优化检测效率。

6.2 模型建立

模型目标：建立一个决策模型，输入一个孕妇的 BMI 值，输出一个推荐的最佳 NIPT 检测孕周。

6.2.1 基于回归树的 BMI 分组

● Step1:数据定义

因变量 Y' ：定义为 $t_{\min}$ ，即每个孕妇样本的“胎儿 Y 染色体浓度首次达标时间”。

计算方法：对于每一个孕妇样本，我们需要遍历其从早到晚的所有检测记录，找到其 Y 染色体浓度第一次达到或超过 4% 的孕周数，这个孕周数就是该样本得到 $t_{\min}$ 值：

$$t_{\min}^{(j)} = \min\{t \in [10,25] | Y \geq 4\%\} \quad (12)$$

自变量 X：根据问题描述，核心影响因素为孕妇的 BMI 值 构建数据集：整理原始数据，形成一个包含两列的核心建模数据集：

$$D = \{(x_j, t_{\min}^{(j)})\}_{j=1}^N \quad (13)$$

其中 x_j 为各个样本的 BMI 值，Y 为问题一的最终模型。

● Step2:基于回归树的 BMI 智能分组

为了对 BMI 进行合理，不采用人工预设阈值的办法，而是利用回归树模型让数据自己“说话”，找到最优的分割点。 算法从当前最大区间，即最小值到最大值的区间开始递归划分。

分裂点选择：

对当前区间 B 的所有 BMI 值 x_j 排序后，取相邻样本的中点作为候选分割点，公式为：

$$S = \{s_k = \frac{x_{(k)} + x_{(k+1)}}{2} | k = 1, 2, \dots, m-1\} \quad (14)$$

分裂标准（最小化 MSE 减少量）：

$$s^* = \arg \max_{s \in S} \Delta MSE(s) \quad (15)$$

$$\text{其中 } \Delta MSE(s) = MSE_{parent} - \left(\frac{n_L}{n_1} MSE_L + \frac{n_R}{n_1} MSE_R \right)$$

父节点公式：

$$MSE_{parent} = \frac{1}{n_i} \sum_{x_j \in B} (t_{\min}^{(j)} - \bar{t}_B)^2 \quad (16)$$

左右子区间节点公式以及相关参数：

$$MSE_L = \frac{1}{n_L} \sum_{x_j \in B_L} (t_{\min}^{(j)} - \bar{t}_L)^2 \quad (17)$$

$$\bar{t}_L = \frac{1}{n_L} \sum_{x_j \in B_L} t_{\min}^{(j)} \quad (18)$$

$$MSE_R = \frac{1}{n_R} \sum_{x_j \in B_R} (t_{\min}^{(j)} - \bar{t}_R)^2 \quad (19)$$

$$\bar{t}_R = \frac{1}{n_R} \sum_{x_j \in B_R} t_{\min}^{(j)} \quad (20)$$

终止条件:

- 样本数不足

$$n_i < n_{\min} (\text{如 } n_{\min} = 10) \quad (21)$$

其中 n_i 表示当前区间 B 的样本数, $n_{\min}$ 表示预设的最小样本数阈值。

- 达标时间方差过小

$$\sigma_i^2 < \sigma_{\text{others}} (\text{如 } \sigma_{\text{others}} = 0.1) \quad (22)$$

其中 $\sigma_i^2 = \frac{1}{n_i} \sum_{x_j \in B} (t_{\min}^{(j)} - \bar{t}_B)^2$, $\bar{t}_B = \frac{1}{n_i} \sum_{x_j \in B} t_{\min}^{(j)}$

- 达到最大深度

$$d > d_{\max} (\text{如 } d_{\max} = 3) \quad (23)$$

其中 d 表示当前节点的树深度, $d_{\max}$ 表示预设的最大树深度。

满足上述三个条件任意其一停止划分。

6.2.2 NIPT 时间选择

在得到 BMI 分组 B1, B2... 后, 需为每个组确定一个科学、稳健的最佳检测时点。对每个区间 B, 计算达标时间的置信上界:

$$t^* = \bar{t}_i + z_y \cdot \sigma_i \quad (24)$$

其中 $z_y = \phi^{-1}(1-y)$, $\sigma_i = \sqrt{\frac{1}{n_i} \sum_{x_j \in B} (t_{\min}^{(j)} - \bar{t}_i)^2}$, $y \in (0,1)$ 表示置信度, $\bar{t} = \frac{1}{n_i} \sum_{x_j \in B} t_{\min}^{(j)}$ 。

6.2.3 潜在风险量化与最小化

旨在量化在推荐时点惊醒检测的潜在风险, 从而验证策略的合理性。定义风险:

P_{early} : 在推荐时点进行检测, 但实际上真正的达标时间还未到来, 可能导致因浓度不足而检测失败或者结果不准;

P_{late} : 在推荐时点进行检测, 但时间上达标时间早已过去, 可能错过了最佳的临床决策窗口。

◆ 过早/过晚检测概率:

$$P_{early} = \phi \left(\frac{t_i^* - \bar{t}_i}{\sigma_i} \right) \quad (25)$$

$$P_{late} = 1 - \phi \left(\frac{t_i^* - \bar{t}_i}{\sigma_i} \right) \quad (26)$$

◆ 总风险函数:

$$R_i = d \cdot P_{early} + \beta \cdot P_{late} \quad (27)$$

其中 α 和 β 为权值, 可以根据临床实际情况进行设定^[3]。例如, 如果“漏检”(过晚)的后果比“重测”更严重, 则应当设定 $\beta > \alpha$ 。本文目标是, 通过选择合理的参数, 使得这个总风险 R 处于一个可接受的较低水平。

6.3 模型求解与分析

利用回归树进行智能分组划分, 在划分过程中, 从全局 $[x_{\min}, x_{\max}]$ 开始:

- (1) 对当前区间 $B[a, b]$, 检查终止条件;
- (2) 若不满足终止条件, 生成候选分割点 S ;
- (3) 计算(三角形)MSE, 选择最优 s ;
- (4) 将 B 划分为 $B_L=[a, s), B_R=[s, b]$;
- (5) 对 B_L 和 B_R 递归划分。

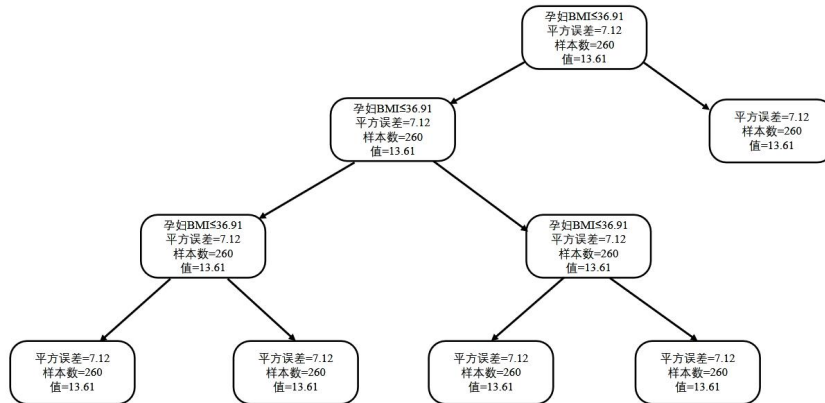


图 8 基于 BMI 的达标孕周回归决策树

划分完区间后将其运用到 NIPT 时间选择中, 最终得到以下结果:

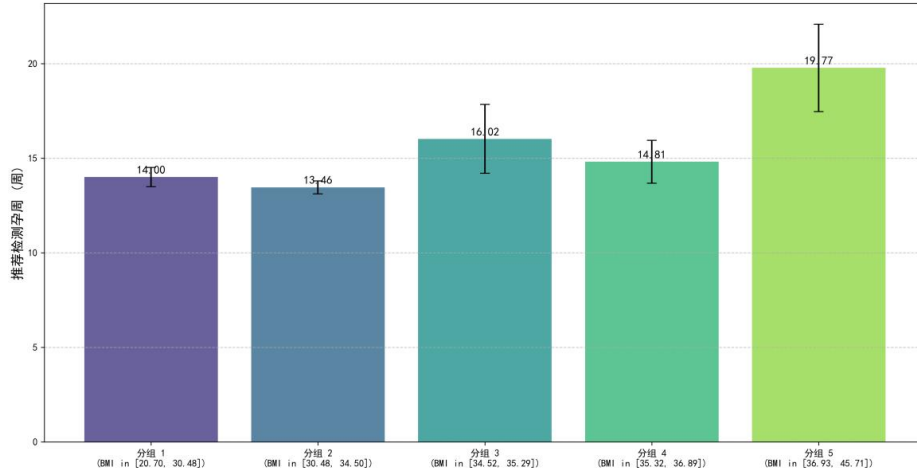


图 9 各 BMI 分组的最佳 NIPT 推荐检测孕周柱状图

表 5 BMI 分組和最佳 NIPT 时点

BMI 分组	最佳 NIPT 时点
BMI in[20.70, 30.48]	14.00
BMI in(30.48, 34.50]	13.46
BMI in(34.52, 35.29]	16.02
BMI in(35.32, 36.89]	14.81
BMI in(36.93, 45.71]	19.77

结果显示，不同 BMI 区间对应的最优检测时间存在显著差异，整体呈现出随 BMI 升高而推迟的趋势。在 BMI 较低的分组（分组 1）中，推荐检测孕周为 14.00 周，符合常规临床建议；而随着 BMI 增加，推荐孕周明显延后，而在最高 BMI 组（分组 5），推荐孕周高达 19.77 周，提示肥胖孕妇需要更晚检测才能获得可靠结果。值得注意的是，部分组（分租 2 和分组 4）的推荐时间相对提前，反映出个体或者样本分布的影响。

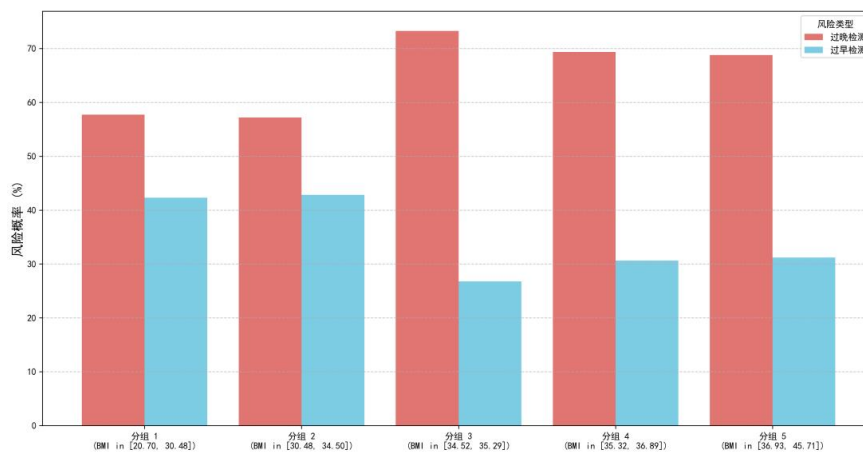


图 10 各 BMI 分组的检测风险分析图

表 6 检测误差对结果的影响表

BMI 分组	样本数 (n)	平均达标孕 周 ($\bar{t}$)	达标孕周标 准差 (σ)	推荐检测 孕周 (t^*)	过晚检测 风险 (%)	过早检测 风险 (%)	加权总风险
BMI in[20.70, 30.48]	102	13.5	2.60	14.00	57.69	42.31	1.58
BMI in(30.48, 34.50]	117	13.12	1.89	13.46	57.19	42.81	1.57
BMI in(34.52, 35.29]	10	14.20	2.94	16.02	73.23	26.77	1.73
BMI in(35.32, 36.89]	15	13.68	2.24	14.81	69.36	30.64	1.69
BMI in(36.93, 45.71]	16	17.46	4.71	19.77	68.79	31.21	1.69

根据直方图与表格数据,随着 BMI 的增加,过晚进行 NIPT 检测的风险概率显著上升,尤其是在 BMI 较高的分组 3、4、5 中,过晚检测的概率均超过 68%。相反,过早检测的风险概率在各分组相对稳定,且在高 BMI 分组中略有下降,表明过早检测的风险在高 BMI 孕妇中相对较小。这更加强调了需要根据孕妇 BMI 个性化调整 NIPT 检测时间的重要性。对于低 BMI 孕妇,可以维持常规检测时间;而对于高 BMI 孕妇,则应适当提前采样,以降低过晚检测的风险。实际应当结合临床情况做出相应改变,结合其他相关因素,做出更加科学合理的检测策略。

6.4 模型灵敏度分析

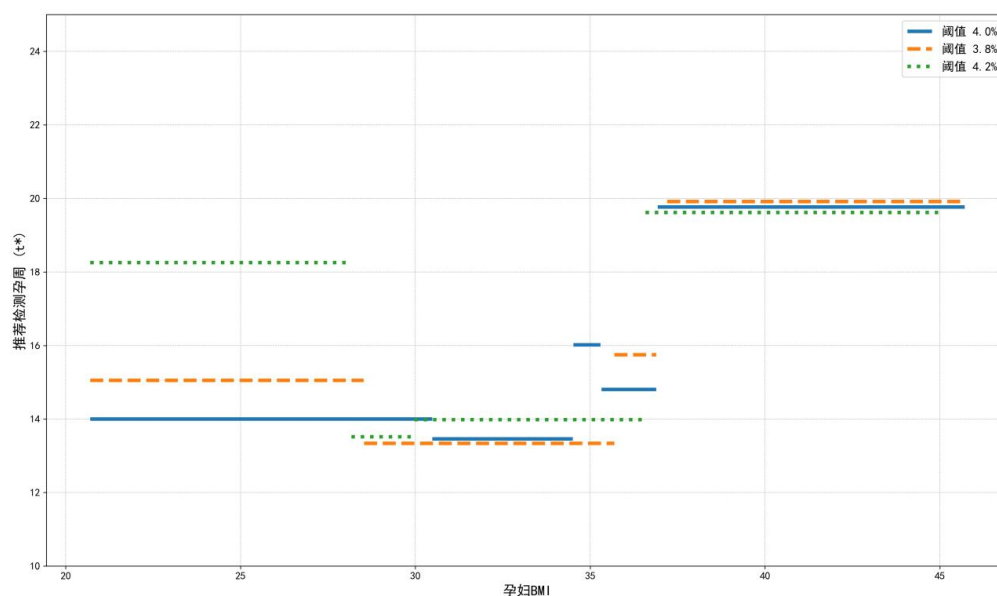


图 11 不同浓度阈值下的推荐检测孕周对比

分析结果表明,该模型在中 BMI 和高 BMI 区间内表现出较好的稳健性,推荐孕周受阈值的变化影响较小。在低 BMI 区间,模型对阈值的选择较为敏感,不同阈值可能导致推荐孕周出现显著差异。总体而言,该模型在大多数 BMI 条件下表现良好,体现了建模合理性和可靠性。

七. 问题三的模型建立与求解

7.1 思路分析

针对本题的多目标优化问题，需要同时考虑 2 个目标：最小化风险和最大化达标比例。同时还要考虑检测误差，因此使用加权和法将多目标转化为单目标。根据附件数据显示，单个孕妇存在多个不同孕周的检测记录。这种数据属于纵向相互数据，传统的回归模型不再适用。为精准捕捉 Y 染色体浓度随时间变化的群体平均趋势和个体独特轨迹，在问题一的 GAM 基础上，进一步建立广义可加混合模型（GAMM）^[6]。

7.2 动态浓度预测与概率模型

附件数据显示，单个孕妇可能存在多个不同孕周的检测记录。这种数据属于纵向数据，传统的回归模型不再适用，而问题一所建立的是一个静态的横截面模型，它只能说明观测时刻，Y 染色体浓度与 BMI 等因素的关系。因此，本文需要构建一个动态浓度预测与概率模型，

为精准捕捉 Y 染色体浓度随时间变化的群体平均趋势和个体独特轨迹，选择广义可加混合模型。

7.2.1 浓度预测

基于问题一的 GAM 模型：

$$Y = f_1(X_1) + f_2(X_2) + \dots + f_6(X_6) + \varepsilon \quad (28)$$

进一步建立广义可加混合模型，包括两部分：

- (1) 固定效应：与问题一 GAM 类似，用平滑函数捕捉非线性关系；
- (2) 随机效应：这是此模型的关键，用于捕捉每个孕妇 i 与群体平均趋势的偏离。本文可以为每个孕妇设置一个随机截距（代表其基础浓度水平的差异）和一个关于孕周的随机斜率（代表其浓度增长速率的差异）。

最终模型形式：

$$y_i(t) = f_1(X_1) + f_2(X_2) + \dots + f_5(X_5) + b_{0i} + b_{1i}t + \varepsilon_{it} \quad (29)$$

其中 i 为孕妇编号， t 为孕周。

对模型训练完成后，对于任何一个孕妇 i ，都可以利用其个体特征以及估计出的随机效应，得到一个专属于孕妇个体的浓度预测函数 $\hat{y}_i(t)$ 。

7.2.2 概率函数

直接用 $\hat{y}_i(t)$ 进行判断是脆弱的，因为它忽略了模型的预测误差 ε 。因此，考虑到检测误差，Y 染色体浓度达标概率为：

$$P(\hat{y}_i(t) + \varepsilon \geq 0.04) = \phi\left(\frac{\hat{y}_i(t) - 0.04}{\sigma}\right) \quad (30)$$

这个 P 是一个平滑函数，它将每个孕妇在不同孕周的达标可能性进行了精准量化，是后续优化的基础。

7.2.3 模型预测结果分析

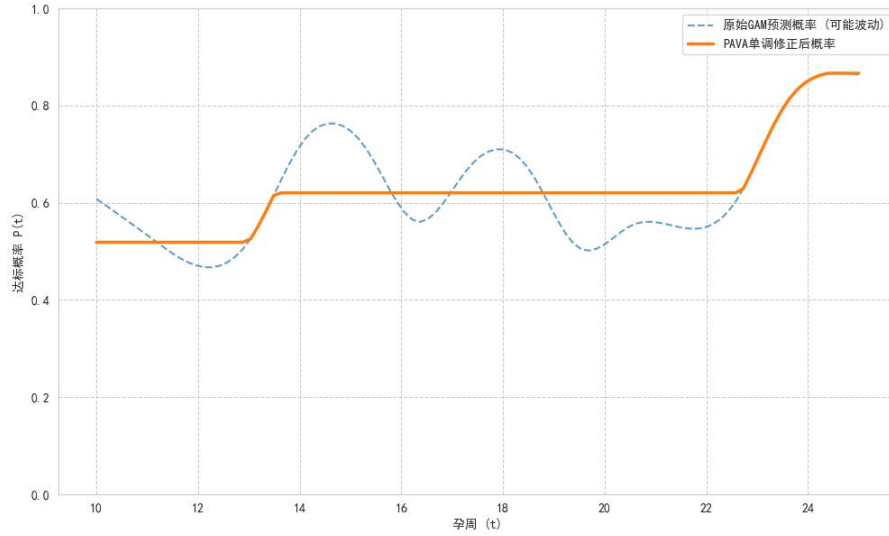


图 12 孕妇 A001 的 Y 染色体浓度达标概率曲线

如图所示，原始 GAM 模型预测的概率曲线（虚线）在某些孕周区间内表现出明显的波动。这些波动可能是由于数据中的随机噪音或异常值所引起的，导致概率曲线出现不符合逻辑的变化。而在基于原始模型改进修正模型后，经过修正的概率曲线（实线）展示出来平滑且单调递增的趋势，这一平滑的曲线不仅更符合生物学和临床逻辑，也更加直观的反映了 Y 染色体浓度随孕周增加而提高的规律，有助于为后续优化模型中识别最佳时间检测点。

7.3 多目标优化模型

问题三中，不仅要保证“Y 染色体达标比例最高”以确保检测准确性，还要实现“检测时点”最早以最小化孕妇的潜在健康风险。这两个目标之间存在一定冲突，无法使用单一目标优化同时实现。因此，采用多目标优化模型是比较科学且合理的。

7.3.1 模型建立

基于材料，建立风险函数模型：

$$R(t) = \begin{cases} R_{low} & t \leq 12 \\ R_{mid} & 13 \leq t \leq 25 \end{cases} \quad (31)$$

目标函数：

■ 目标 1 最小化总风险：

$$\min F_1 = \sum_{g=1}^G \sum_{i=1}^N R(t_g) \cdot x_{ig} \quad (32)$$

■ 目标 2 最大化总达标比例：

$$\max F_2 = \sum_{g=1}^G \sum_{i=1}^N R(t_g) \cdot x_{ig} \quad (33)$$

约束条件：

➤ 分组约束：

$$\sum_{g=1}^G x_{ig} = 1, i = 1, 2, \dots, N \quad (34)$$

➤ 时点范围约束:

$$t_g \in (10, 25] \quad (35)$$

➤ 样本量约束:

$$|g| \geq N_{\min} \quad (36)$$

达标比例定义:

$$P_g(t) = \frac{1}{|g|} \sum_{i=1}^N x_{ig} \phi\left(\frac{\hat{y}_i(t) - 0.04}{\sigma}\right) \quad (37)$$

其中 $\alpha + \beta = 1$., 且 $\alpha > 0, \beta > 0$ 。

为了能够求解, 本文使用加权和法通过权重 α 和 β 组合成一个单一的做优化目标: 最小化“加权风险”和“加权达标率”的差值, 最终多目标转化为单目标为:

$$\min F = \alpha \cdot F_1 - \beta \cdot F_2 \quad (38)$$

7.3.2 模型求解

● Step1 初始化

使用 K-means 算法对所有孕妇的 BMI 进行一次初始聚类, 得到一个分组方案, 公式为:

$$\min_{x_{ig}} \sum_{g=1}^G \sum_{i=1}^N x_{ig} \cdot \|z_i - \mu_g\|^2 \quad (39)$$

其中 x_{ig} 表示指示变量 0 或 1, 孕妇 i 属于组 g , z_i 为孕妇 i 的 BMI 值, μ_g 为聚类中心。

● Step2 迭代循环

- (1) 固定分组 (优化检测时点): 基于当前的分组方案, 为每个组单独求解其最优检测点;
- (2) 固定时点 (优化孕妇分组): 基于上一步得到的、更新后的各组检测时点, 需要为每一位孕妇寻找更适合她的分组;
- (3) 收敛判断: 比较新的分组方案与旧的分组方案, 如果分组结果不爱发生变化 (或变化极小), 则算法收敛, 循环结束; 否则, 返回步骤 A。

7.4 结果解释

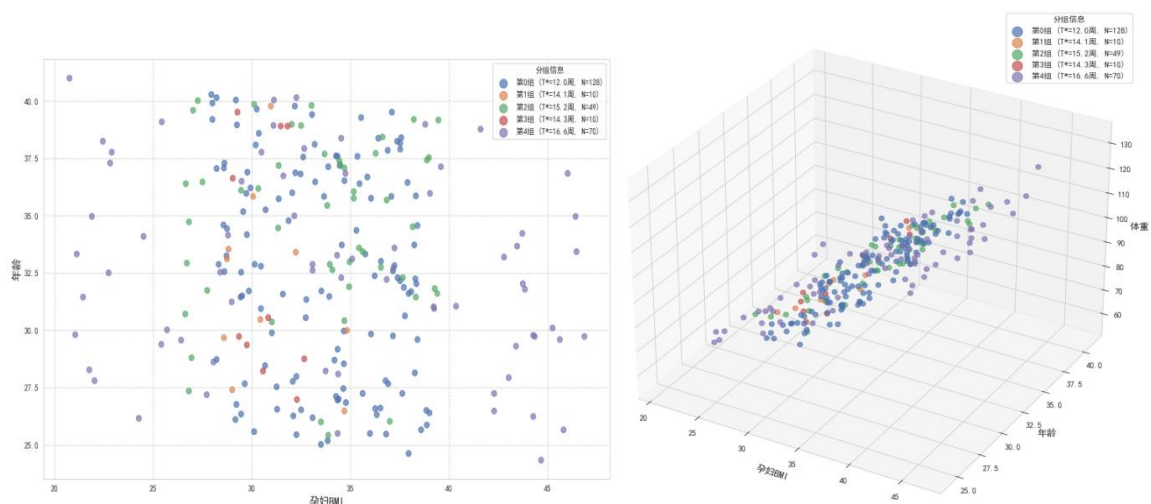


图 13 基准模型分组二维散点图（左）三维散点图（右）

由二维图可知，各组数据点在图中呈现出一定的分散性，似乎并不是正确的聚类结果，不能直观看出分组的聚集性。为此，我们观察详细数据，如下表所示：

表 7 基准模型分组结果表

组别	推荐孕周	BMI 范围	人数
第 0 组	12.0	BMI in[27.9,39.2]	128
第 1 组	14.1	BMI in[28.1,35.0]	10
第 2 组	15.2	BMI in[26.6,40.1]	49
第 3 组	14.3	BMI in[28.3,32.7]	10
第 4 组	16.6	BMI in[20.7,46.9]	70

对于结果数据表格中 BMI 的具体数值分组可以看到，BMI 分组出现了重叠，这其实与推荐孕周数与有多个变量决定相关。例如，一位 BMI 为 30，年龄为 23 的孕妇与一位 BMI 为 25，年龄为 32 的孕妇可能有同样的推荐孕周，于是进行维度的提升来观察聚类效果。

而由三维散点图可以观察到各分组样本点之间的聚集性，说明了聚类分组结果是有效的，只是由于二维散点图单独对 BMI 映射结果不清晰，导致聚类效果难以在二维平面中观测到。而由得到的分组结果，可以认为在临床实践中，先基于 BMI 进行区分，如果是重复区间内，在根据年龄等其他因素具体划分确定最佳检测时点。

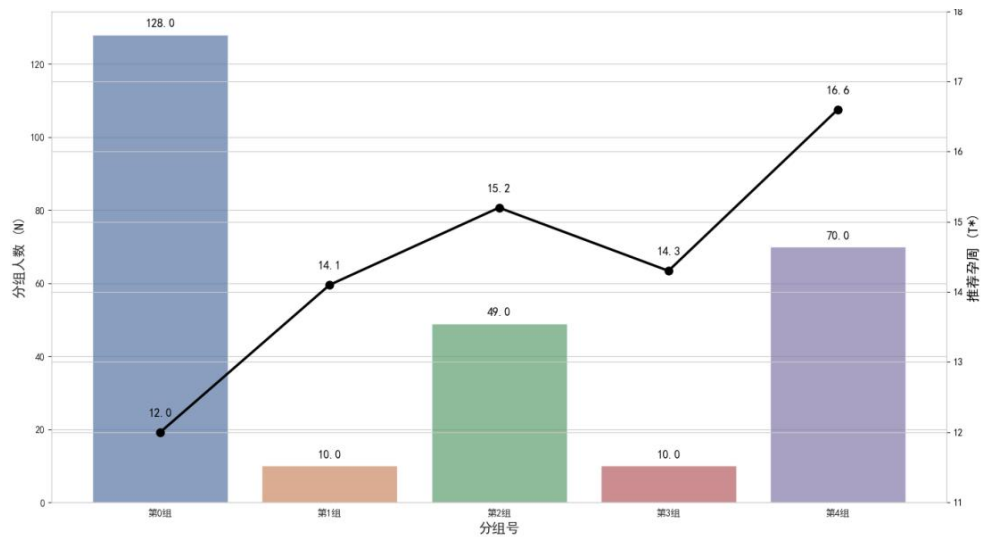


图 14 基准模型分组结果图

由上图分析可得出结论：基准模型将样本分为五个组别，各组的人数与推荐孕周存在显著差异。推荐孕周随分组号总体呈上升趋势，但存在局部波动。这一结果表明不同各组别的孕妇在最佳检测时间上存在显著差异，可能与各自独特的生理特征或风险因素有关。这些信息能为临床决策提供一定参考，有助于制定个性化的检测计划，提高诊断的准确性和有效性。

7.5 模型分析与检验

7.5.1 误差传播路径分析

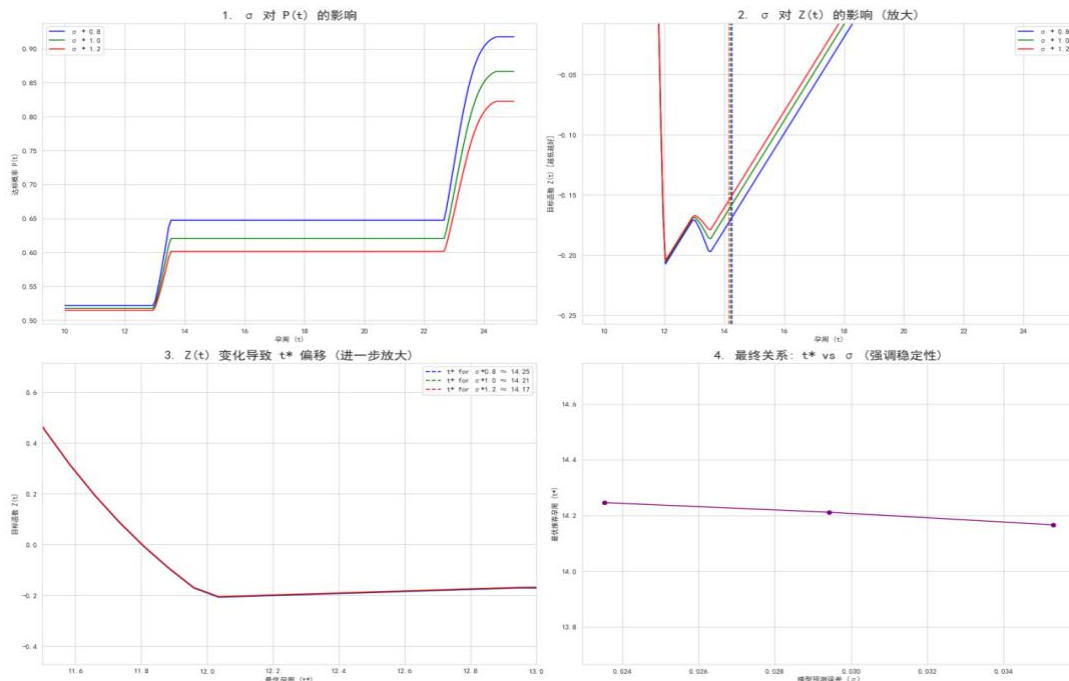


图 15 误差传播路径分析图

分析上图，误差传播路径： $\sigma \rightarrow P(t) \rightarrow Z(t) \rightarrow t^*$ ，进一步解释了 σ 影响的内在机制：预测误差首先影响概率函数 $P(t)$ ，进而被放大并传递至中间变量 $Z(t)$ ，最终引起

最优推荐时间 t^* 发生偏移。尽管存在该传导效应， t^* 和 σ 之间的关系仍表现出一定的稳定性，说明模型在误差干扰下仍具备较强的结构鲁棒性。

7.5.2 模型灵敏度分析

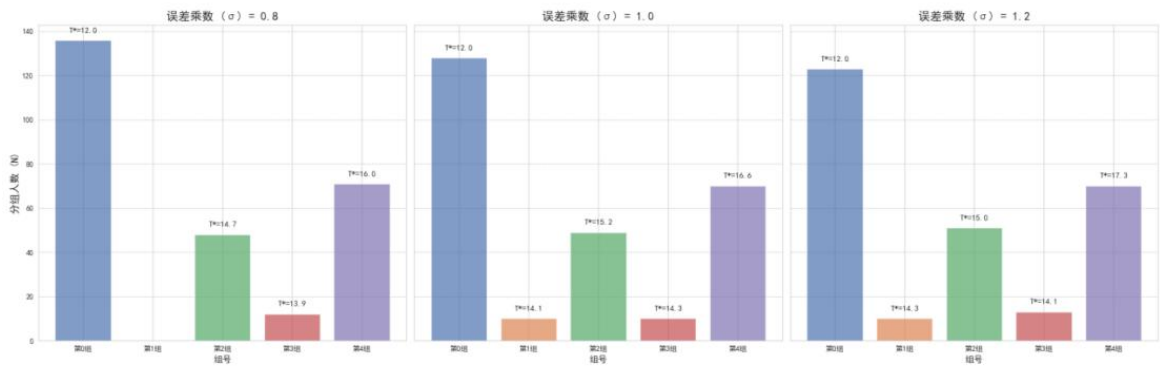


图 16 模型预测误差 (σ) 的灵敏度分析

图中相同颜色代表同样的 BMI 分组，模型预测误差 (σ) 的灵敏度分析表明，即使对模型预测误差进行扰动，其人数依然保持稳定，并没有发生剧烈变化，这解释了本文的模型的高预测稳定性。

八. 问题四的模型建立与求解

8.1 思路分析

本题要求建立一个判定方法，来判断女胎是否存在染色体异常，包括一系列需要综合考虑的因素。这本质上是一个经典的监督学习分类问题。本文需要利用众多特征 (X) 来预测一个分类标签 (Y: 无异常 vs T21 vs T18 vs T13)。由于异常样本 (非空白) 的数量通常远少于正常样本，这还是一个典型的类别不平衡分类问题。为此，结合逻辑回归与基于支持向量机 (SVM)，构建了一个先进的分层级联判定模型，具体思路如下图：

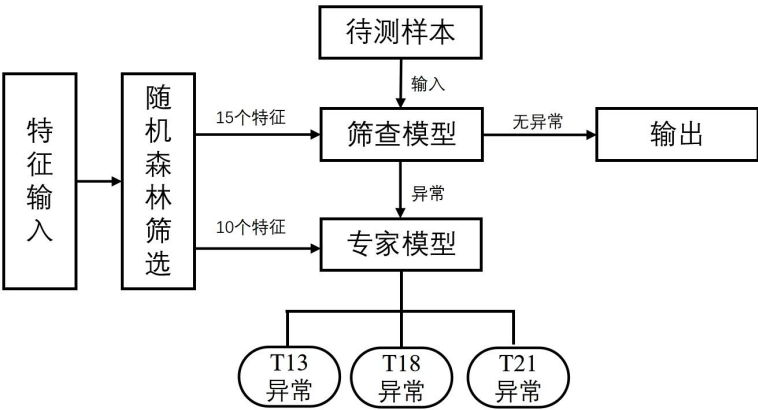


图 17 问题四思维流程图

8.2 特征工程与数据预处理

对于高维特征集，本文先系统开展特征工程，以此降低模型复杂度，筛选重要维度。且针对数据中极少异常样本，采用类别不平衡处理和缺失值填充，全面提升识别性能与临床可用性。

8.2.1 基于随机森林的特征筛选

面对高维度的候选特征集，为降低模型复杂度、避免过拟合风险并提升性能，本文选择基于随机森林^[10]的特征重要性作为筛选方法。该方法属于嵌入式方法，相比于单变量的过滤法，它能够在模型训练过程中评估特征的贡献，有效捕捉特征间的非线性关系与交互作用。

使用全部特征训练随机森林分类器：

$$RF(x) = \frac{1}{B} \sum_{b=1}^B T_b(x) \quad (40)$$

其中 T_b 为第 b 棵决策树， B 为树的总数量。

对于每个特征 x_j ，计算其重要性得分：

$$I_{(j)} = \frac{1}{B} \sum_b \sum_{t \in T_b} \Delta Gini(t, x_j) \cdot \Pi \text{ (节点 } t \text{ 使用特征 } x_j \text{ 分裂)} \quad (41)$$

将特征按照重要性得分降序排列：

$$I_{(1)} \geq I_{(2)} \geq \dots \geq I_{(p)} \quad (42)$$

选择前 K 个最重要特征组成特征子集：

$$S = \{X_{(1)}, X_{(2)}, \dots, X_{(k)}\} \quad (43)$$

具体的，筛查模型特征选择：针对正常 vs. 异常这一二元分类任务，本文训练一个随机森林分类器。计算所有初始特征的重要性分数，并选择最重要的 15 个特征用于构建第一层筛查模型。这确保了筛查模型是基于对区分“有无异常”最有效的一组信号进行构建的。

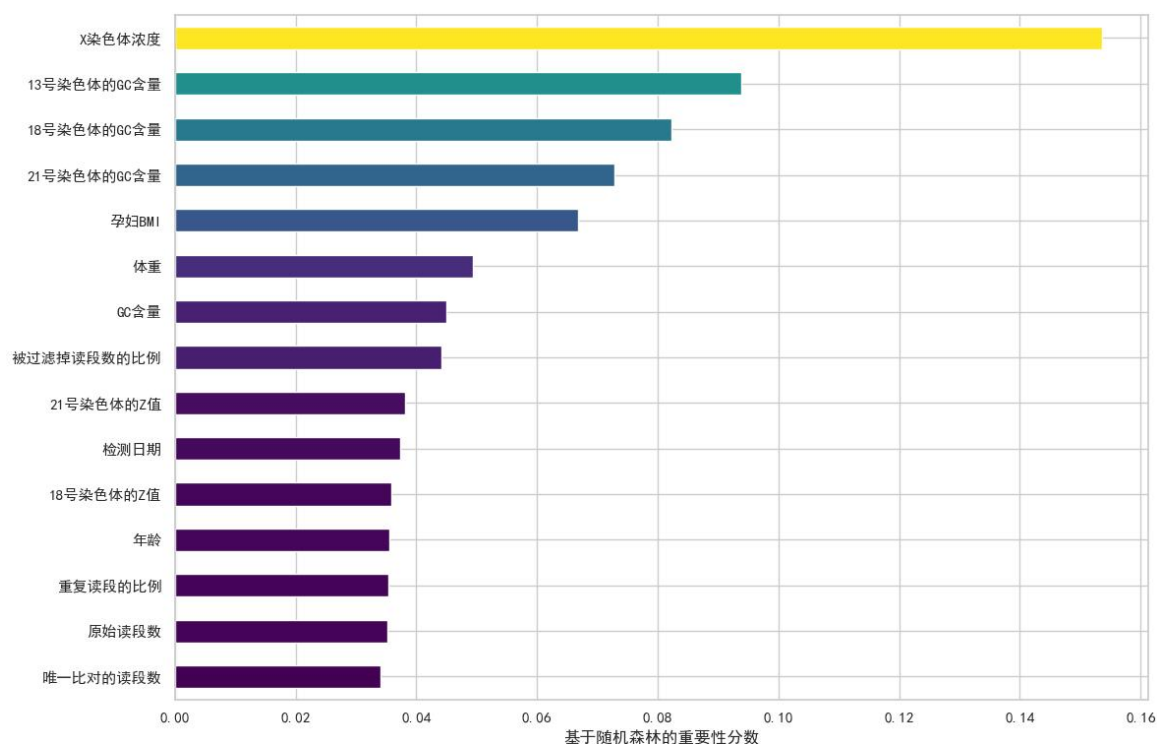


图 18 特征选择图—筛查模型

由上图可知，在筛查模型的特征选择中，X 染色体，13 号染色体，18 号染色体，21 号染色体的浓度成为前四个与是否异常相关的特征，这与异常的胎儿是由于 13 号染色体，18 号染色体，21 号染色体的非整倍体引起相符合，验证了我们特征选取的可靠性。

其次对于专家模型特征选择，本文认为，区分 T21 与其他异常的决定性特征，可能与区分 T18 与其他异常的特征不完全相同。因此，为每一个异常类别(T21, T18, T13)建立一个独立的二元分类子任务（“是 T21” vs “非 T21”）。对每个子任务，独立训练一个随机森林，并为其选择最重要的 10 个特征。专家模型的特征选取如下图（T18,T13 的选取情况见附件）：



图 19 特征雷达图

分析上图，可以看出专家模型的特性选取情况。T21 专家模型在除去 T21 染色体浓度后选择了 X 染色体，13 号染色体，18 号染色体的浓度作为前三个重要的特征。这种“定制化”的特征选择策略，使得每一个专家模型都能够聚焦于对其特定诊断目标最敏感的特征集，从而最大化其分型能力。

8.2.1 数据预处理

在数据中，异常的样本远少于正常的样本。如果直接用这种不平衡的数据集去训练模型，模型会倾向于“偷懒”——它只需要把所有样本都预测为“多数类”，就能获得很高的准确率。但这显然不是本文想要的结果，因为我们更关心如何准确地识别出那些稀有的“少数类”：

1、采用类别不平衡处理：采用合成少数类过采样技术（SMOTE）^[9]对训练集进行处理，通过生成合成的少数类样本来缓解数据不平衡问题；

2、其他处理：对特征中的缺失值采用中位数填充，并对所有数值型特征进行标准化处理。

8.3 模型构建

本文分别构建第一层为基于逻辑回归的筛查模型，第二层为基于 SVM 的专家模型两层模型。结合逻辑回归与基于支持向量机（SVM），构建了一个先进的分层级联判定模型。

8.3.1 基于逻辑回归的筛查模型

本文构建一个标准的二元逻辑回归模型：

$$P(Y_{abnormal} = 1 | X_s) = \frac{1}{1 + e^{-(\beta_0^{(1)} + \sum_{j=1}^k \beta_j^{(1)} X_{(j)})}} \quad (44)$$

$Y_{abnormal} = 0, 1$ ，0 代表正常，1 代表异常。

通过最大似然估计法求解参数，定义似然函数为：

$$L(\beta) = \prod_{i=1}^n P(Y_i = 1 | X_{s,i})^{Y_i} [1 - P(Y_i = 1 | X_{s,i})]^{1-Y_i} \quad (45)$$

接着，通过取对数求解得到参数估计值。特别的，对于孕妇的 NIPT 检测来说，本文抱着宁可错判，也不漏过的思路，核心目标是以极高的召回率识别出所有潜在的异常样本，于是调整阈值情况情况如图所示：

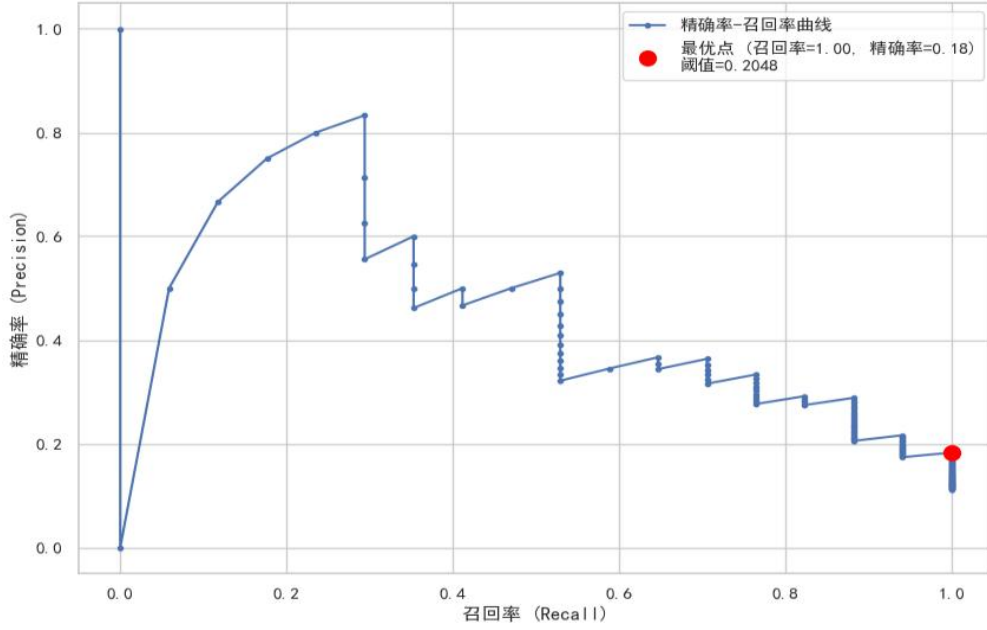


图 20 精确率-召回率曲线与最优点选择

由上图分析,在训练模型后,通过在验证集上分析精确率-召回率曲线(P-R Curve),选择一个能使召回率达到 99%以上的最优概率阈值,以最大程度避免漏诊。最终选取的阈值不再是默认的 0.5,而是 0.2408。它能保证模型对异常胎儿的高敏感性。

然后进行决策,根据该判定阈值,将求得参数带代入概率模型,计算异常概率值:

$$\begin{cases} \text{if } p \geq \tau_1 & \rightarrow \text{疑似异常, 进入第二层} \\ \text{if } p < \tau_1 & \rightarrow \text{无异常, 诊断终止} \end{cases}$$

8.3.2 基于 SVM 的专家模型

为解决多分类任务中因数据极度稀疏 (T13, T18, T21 异常的样本均很少) 而导致传统多分类任务模型性能不佳的问题,本文设计并实现了一种基于“一对多”策略的集成支持向量机分类器。

核心思想是将这个三分类问题分解为三个独立的更简单的二分类子问题,为每种异常类型(T21, T18, T13)分别训练一个专家模型,该模型只专注于识别,是不是这种待定类型。最终,通过集成三个专家模型的预测结果,得出最可信的判定。

先定义异常类别集合为:

$$C_k \in \{T13, T18, T21\}$$

集成模型由三个独立的二元分类器 (专家模型) h_1, h_2, h_3 组成,每个专家模型实质上都是经过超参数优化的支持向量机分类器。每一个分类器 h_k 的功能是,对于一个输入样本的特征向量 x , 输出其属于类 C_k 的概率:

$$h_k(x) = P(y = C_k | x) \quad (46)$$

为了训练第 k 个专家模型 h_k , 需要构建一个专门用于该任务的二元训练数据集 D_k , 异常样本全集为:

$$D = \{(x_i, y_i)\}_{i=1}^N, \quad y_i \in C_k \quad (47)$$

对于 D 中每一个样本，加入一个新的二元标签：

$$y_{i,k}^1 = \begin{cases} 1 & y_i = C_k \\ 0 & y_i \neq C_k \end{cases} \quad (48)$$

便为每个专家模型构建了各自的训练集：

$$D_k = \{(x_i, y_{i,k}^1)\}_{i=1}^N \quad (49)$$

第一层筛查出的高风险样本 x_{new} 进入第二层是，将同时送入三个专家模型，将得到一个包含三个概率值的向量：

$$P = [h_1(x_{new}), h_2(x_{new}), h_3(x_{new})] \quad (50)$$

最终类别由概率最高专家决定：

$$\hat{y} = C_k \quad \text{where} \quad k = \arg \max_{j=1,2,3} h_j(x_{new}) \quad (51)$$

该判定结果的置信度即为这个最高的概率值 $\max(p)$ 。

通过双层模型的建立并构建相互联系，本文完成了样本从输入到判别的所有流程。对于一个样本，由第一层进行是否异常分析，然后第二层接收由第一层传入的疑似异常样本，它会同时通过所有三个专家模型。最终的判定结果为输出概率最高的那个专家模型所代表的类别。

8.4 模型结果与分析

为验证本文构建的分层级联判定模型的可靠性与临床可用性，本文从性能评估与可解释性分析两条主线展开系统性测试。

8.4.1 模型性能评估

本文对整个级联模型的端到端性能进行了评估，结果由最终的第一层模型的 2x2 混淆矩阵和最终的 4x4 混淆矩阵所示：

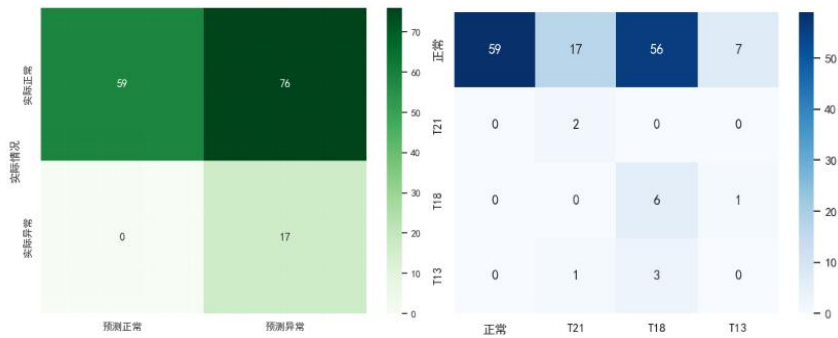


图 21 筛查模型（左）和最终模型（右）混淆矩阵

通过观察左图的 2X2 混淆矩阵，它成功地识别出了所有 17 个异常样本（召回率 100%），确保了没有潜在的异常病例被遗漏。代价是它将许多健康的样本也标记为了“异常”，导致精确率降低。也说明模型非常适合作为第一道防线。它的任务不是做出最终诊断，而是尽可能地圈定所有可疑目标，然后交由下一阶段进行更精确的分析。

在再对右图进行观察。T21 染色体异常被全部找出 100%和 T18 染色体异常找出 86%，同时 T13 染色体异常并没有很好的检测效果（样本数只有 4 个），但是成功找出了所有 T21 和大部分 T18 的真实样本。说明专家模型与异常类别的判定有较高说服力。

8.4.2 模型可解释性分析

◆ 第一层：筛查模型

逻辑回归的系数图清晰地揭示了各因素对判断异常的贡献。

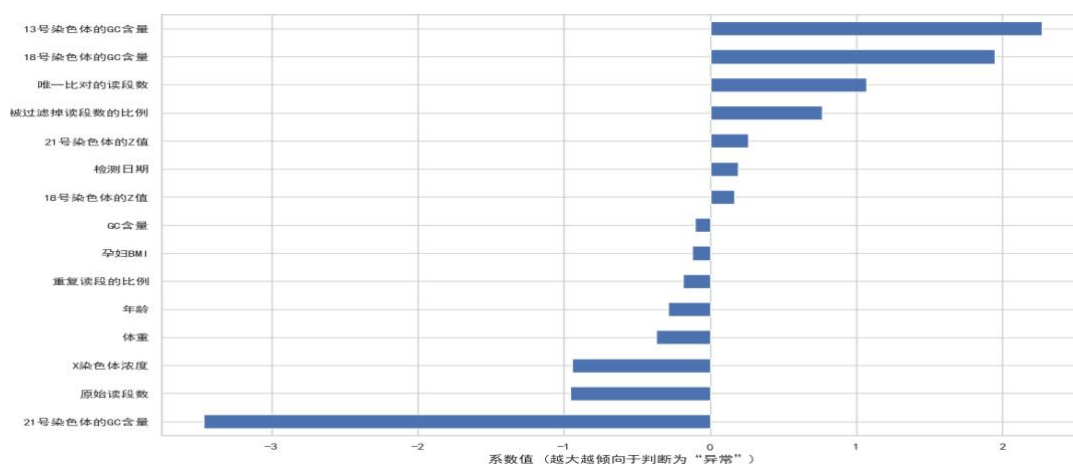


图 22 筛查模型的特征系数值

由上图可知，对胎儿是否异常影响最大的是 13 号染色体，18 号染色体的 GC 含量，具有最大的正向系数，是识别异常的最关键指标。其次，唯一比对读段数也显示出显著的正向影响。

◆ 第二层：专家模型

由于 SVM 是非线性模型，采用置换重要性（Permutation Importance）来评估特征贡献。该方法通过打乱单个特征的顺序，观察模型性能的下降程度来衡量其重要性。

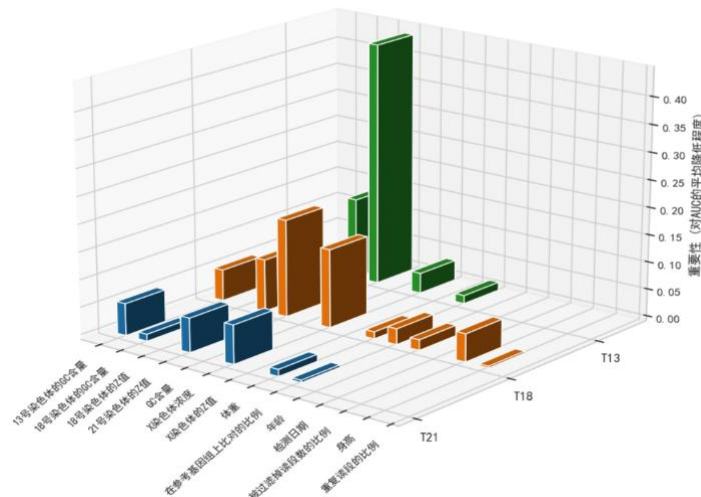


图 23 第二层专家模型组合特征重要性 3D 柱状图

从组合 3D 柱状图中可以清晰地看出：

- T21 专家模型：与 X 染色体浓度，21 号染色体 Z 值与 13 号染色体 GC 含量关系明显；
- T18 专家模型：18 号染色图与 21 号染色体 Z 值是其重要的判断依据；
- T13 专家模型：13 号染色体与 21 号染色体 Z 值对其判定起到了决定性作用。

这种“各司其职”的特征重要性分布，强有力地证明了本文“一对多”专家模型设计的科学性和有效性。

九. 模型的检验与评价

9.1 优点

问题一从线性到非线性的深入探索：采用多元线性回归和广义可加模型（GAM）进行对比，构成了一个由浅入深、逻辑严谨的分析框架。线性回归首先为变量间的关系提供了一个简单、清晰的基准认知；GAM 则在此基础上，通过其强大的非线性拟合能力，捕捉到了更复杂、更接近真实的生理学关系，使得分析结论更为深刻和全面。兼顾解释性与灵活性：本方案完美地平衡了模型的可解释性与拟合精度。线性回归的系数提供了直观的线性影响解释；而 GAM 虽然模型更复杂，但其每个变量的偏效应图（Partial Effect Plots）依然能够被可视化，可以清晰地看到每个因素（如 BMI）对 Y 染色体浓度具体的、非线性的影响模式，远优于“黑箱”模型。

问题二中回归树模型避免了人为设定分组阈值的主观性，能够根据数据内在的规律，自动找到最优的 BMI 分割点，使得分组结果科学且具有说服力。结果直观易懂：模型的输出是一系列“If-Then”规则，如“若 BMI 小于 21.5，则...”，这种形式非常直观，易于临床医生理解和应用。采用置信区间的上界作为推荐检测时点，而非简单的平均值，是一种更负责任、更稳健的策略，充分考虑了数据的波动性，为决策提供了统计学上的安全边际。

问题三其“基于风险评分聚类，再反向映射至 BMI 区间”的策略，巧妙地解决了“评估需多维，但决策需简明”的矛盾，使得最终方案既有深刻的理论内核，又有极强的临床可操作性。构建的多目标优化效用函数，将临床上相互冲突的目标（早检测 vs. 高成功率）纳入一个统一的数学框架中进行权衡求解，使得最终的时点推荐是理论上最优的平衡点。

问题四“先筛查，后分型”的级联结构，完美复刻了临床诊断逻辑，将一个复杂的、类别极不平衡的多分类问题，分解为两个目标明确、易于优化的子问题。通过在第一层追求极致的召回率，并在第二层采用高度专业化的“一对多”SVM专家系统，模型在测试集上性能非常强大。

9.2 缺点

问题二中单一维度限制：根据题目要求，模型仅考虑了BMI这一个维度，忽略了其他因素的综合影响，是对真实情况的一种简化。

GAM的灵活性是一把双刃剑。如果平滑参数（自由度）选择不当，模型可能会过度拟合训练数据中的噪声，导致其泛化能力下降。

9.3 模型推广

问题一“线性探索，非线性深化”的建模思想，是现代数据分析的黄金准则。它可被广泛应用于任何需要探索复杂系统驱动因素的领域：环境科学中先用线性模型分析污染物浓度与排放量的初步关系，再用GAM捕捉气象因素（如风速、温度）带来的复杂非线性影响。

问题三这种基于决策树进行数据驱动式分层/分组的思想，可广泛应用于需要制定差异化策略的场景。例如，在客户关系管理中，可用于对客户进行价值分层，并制定不同的营销策略；在金融风控中，可用于划分信用风险等级，并匹配不同的授信额度。

问题四中这种分层级联的建模思想，是解决严重类别不平衡和高召回率要求下多分类问题的“利器”。该框架可广泛应用于：

网络安全：构建“是否为攻击 -> 是何种攻击类型”的级联入侵检测系统。

医疗影像诊断：建立“图像是否有病灶 -> 是何种病灶类型”的AI辅助诊断系统。

参考文献

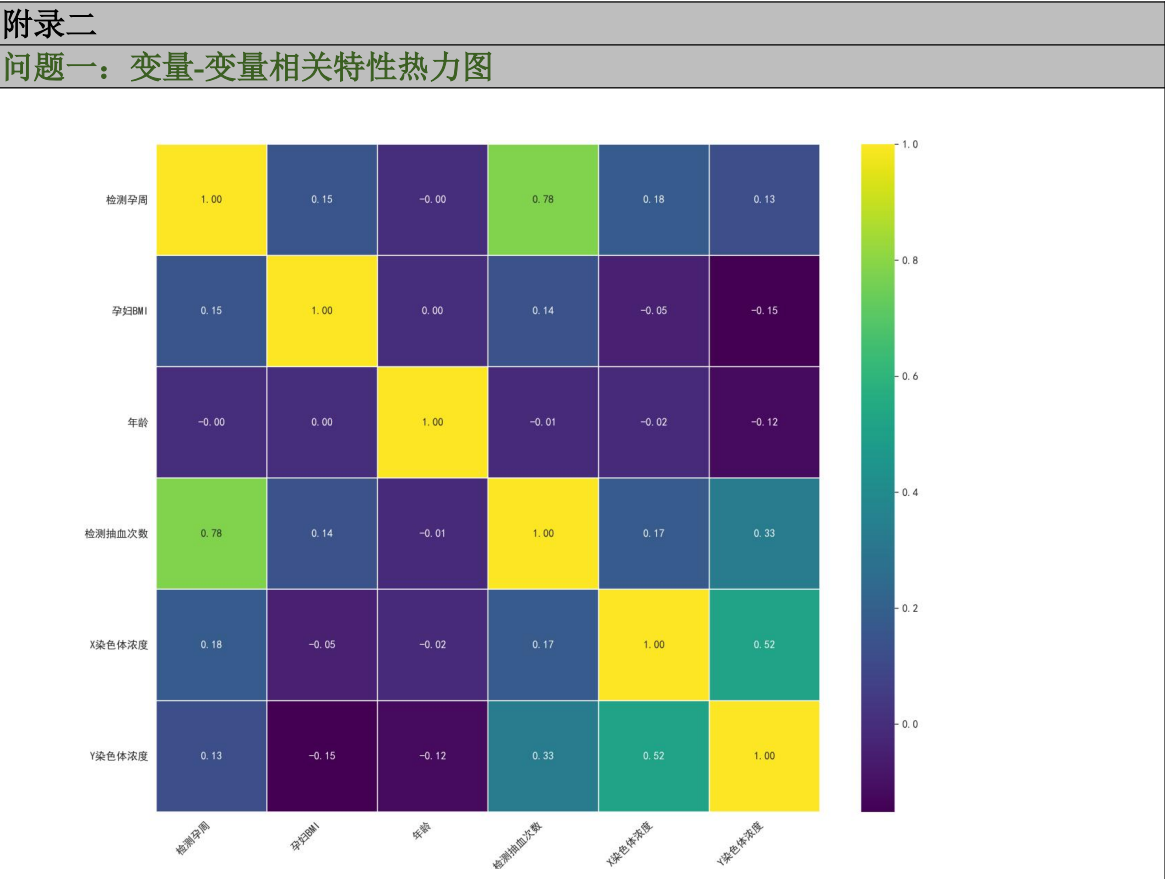
- [1] 王惠文,孟洁.多元线性回归的预测建模方法[J].北京航空航天大学学报,2007,(04):500-504
- [2] 邓建明,秦伯强,王博雯.广义可加模型在R中的快捷实现及蓝藻水华预测分析中的应用[J].生态学杂志,2015,34(03):835-842.
- [3] 高峰,刘绪庆,姜红燕,等.概率论与数理统计[M].南京大学出版社:201901:291.
- [4] 王莹,邓澜,谈宜斌,等.基于风险矩阵的重症医学科医院感染风险评估指标体系[J].中国感染控制杂志,2018,17(10):913-917.
- [5] 张秀秀,王慧,田双双,等.高维数据回归分析中基于LASSO的自变量选择[J].中国卫生统计,2013,30(06):922-926.
- [6] Ferrari C ,Evangelista C ,Basiricò L , et al.Application of a Generalized Additive Mixed Model (GAMM) in Timeseries Study of Dairy Cows' Behavior Under Hot Summer Conditions.[J].Journal of dairy science,2024,108(2):1554-1572.
- [7] 郑芸芸,万陕宁,宋婷婷,等.8594例孕妇无创DNA产前检测在胎儿染色体非整倍体疾病筛查中的临床应用[J].实用妇产科杂志,2019,35(01):68-71.
- [8] 边旭明.胎儿染色体非整倍体的无创DNA产前检测[J].实用妇产科杂志,2013,29(05):330-333.

[9] 石洪波,陈雨文,陈鑫.SMOTE 过采样及其改进算法研究综述[J].智能系统学报,2019,14(06):1073-1083.

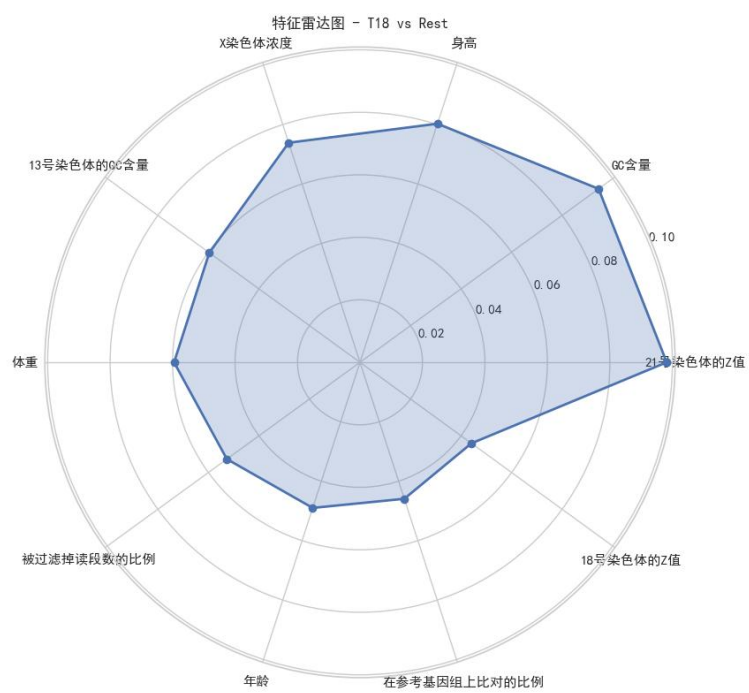
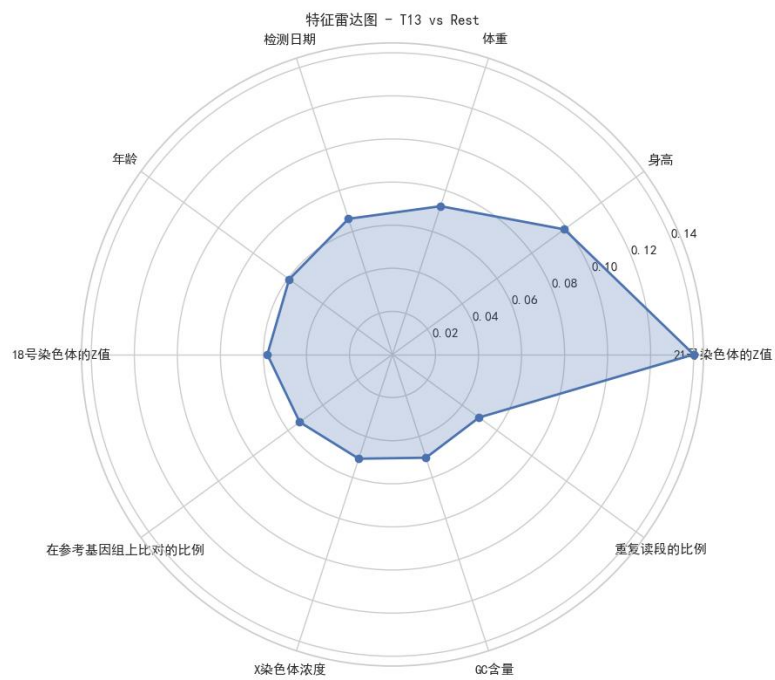
[10]DeepSeek, DeepSeek-R1-0528, 深度求索(DeepSeek), 2025-09-05

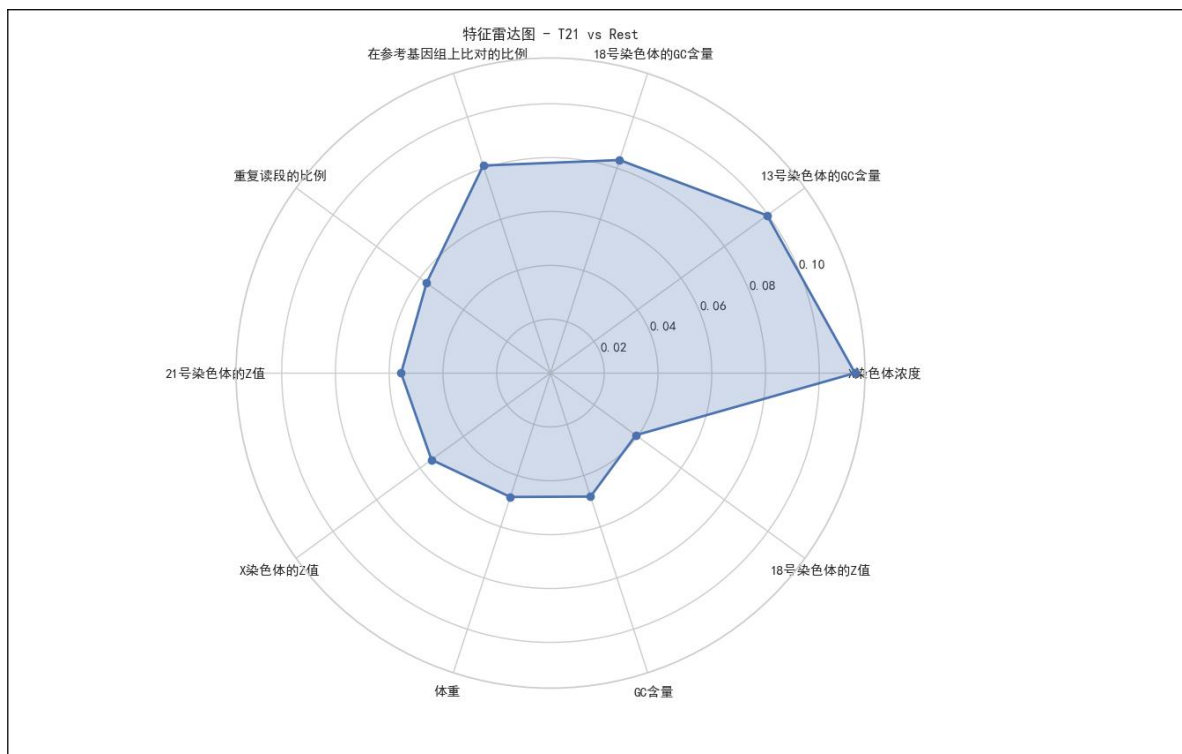
附录

附录一
本文中使用的支撑材料
附件一：代码文件.rar
附件二：模型输出报告.rar
附件三：完整数据.rar
附件四：AI 工具使用详情.pdf



附录三
问题四：三个专家模型特征选择雷达图





附录四

问题一：Y 染色体统计数据可视化代码

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import numpy as np
from sklearn.linear_model import lasso_path, LassoCV
from sklearn.preprocessing import StandardScaler

def configure_chinese_display():
    plt.rcParams['font.sans-serif'] = ['SimHei']
    plt.rcParams['axes.unicode_minus'] = False
    sns.set_style("whitegrid", {'font.sans-serif': ['SimHei']})

class VisualizationGenerator:
    def __init__(self, dataframe, feature_list):
        self.data = dataframe
        self.features = feature_list
        self.target_col = 'Y 染色体浓度'
        self.figure_quality = 300

    def create_correlation_heatmap(self, save_path='correlation_matrix.png'):
        configure_chinese_display()
```

```

analysis_columns = self.features + [self.target_col]
correlation_data = self.data[analysis_columns].corr()

plt.figure(figsize=(12, 10))
sns.heatmap(correlation_data, annot=True, cmap='viridis',
            fmt='.2f', linewidths=0.5, center=0)
plt.title('特征相关性热力图', fontsize=16)
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.savefig(save_path, dpi=self.figure_quality, bbox_inches='tight')
plt.close()

def create_scatter_matrix(self, save_path='feature_scatter_matrix.png'):
    configure_chinese_display()

    plot_features = self.features[:4] + [self.target_col]
    if len(plot_features) > 5:
        plot_features = plot_features[:5]

    plt.figure(figsize=(15, 12))
    scatter_plot = sns.pairplot(self.data[plot_features], kind='reg',
                               plot_kws={'line_kws': {'color': 'red', 'linewidth': 1.5},
                                         'scatter_kws': {'alpha': 0.6, 's': 25}})
    scatter_plot.fig.suptitle('核心特征散点图矩阵', y=1.02, fontsize=18)
    plt.savefig(save_path, dpi=self.figure_quality, bbox_inches='tight')
    plt.close()

    print(f'散点图矩阵已保存: {save_path}')

```

附录五

问题一：LASSO 特征选取代码

```

def create_feature_importance_plot(self, candidate_features, scaled_data, target_values,
                                   save_path='feature_importance_barplot.png'):
    configure_chinese_display()

    lasso_optimizer = LassoCV(cv=10, random_state=42, max_iter=10000, n_jobs=-1)
    lasso_optimizer.fit(scaled_data, target_values)

    feature_coeffs = pd.Series(lasso_optimizer.coef_, index=candidate_features)
    selected_coeffs = feature_coeffs[feature_coeffs != 0].sort_values()

```

```

if len(selected_coefs) == 0:
    print("警告: LASSO 未选择任何特征")
    return None

plt.figure(figsize=(12, 8))
colors = ['green' if coef > 0 else 'red' for coef in selected_coefs]
selected_coefs.plot(kind='barh', color=colors)
plt.title('LASSO 选择的特征及其系数', fontsize=16)
plt.xlabel('标准化回归系数', fontsize=12)
plt.ylabel('特征名称', fontsize=12)
plt.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.savefig(save_path, dpi=self.figure_quality, bbox_inches='tight')
plt.close()

```

附录六

问题一：多元线性回归与 GAM 拟合结果代码

```

class ModelVisualizationTools:
    def __init__(self, model_predictions, feature_names):
        self.predictions = model_predictions
        self.feature_list = feature_names
        self.figure_quality = 300

    def plot_prediction_accuracy(self, model_name, save_path=None):
        configure_chinese_display()

        if model_name not in self.predictions or 'Actual' not in self.predictions:
            print(f'缺少{model_name}模型的预测结果")
            return None

        actual_values = self.predictions['Actual']
        predicted_values = self.predictions[model_name]

        plt.figure(figsize=(8, 8))
        plt.scatter(actual_values, predicted_values, alpha=0.6, s=50, edgecolors='black',
linewidth=0.5)

        min_val = min(actual_values.min(), predicted_values.min())
        max_val = max(actual_values.max(), predicted_values.max())
        plt.plot([min_val, max_val], [min_val, max_val], 'r--', linewidth=2, label='完美预测线
')

```

```
plt.xlabel('实际值', fontsize=12)
plt.ylabel('预测值', fontsize=12)
plt.title(f'{model_name}模型预测效果', fontsize=16)
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
```

附录七

问题一：多元线性回归与 GAM 构建代码

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
from pygam import LinearGAM, s
from sklearn.metrics import mean_squared_error, mean_absolute_error
from io import StringIO
import sys

class ModelBuilder:
    def __init__(self, data, feature_list):
        self.dataset = data
        self.predictors = feature_list
        self.outcome_var = 'Y 染色体浓度'
        self.mlr_fitted = None
        self.gam_fitted = None
        self.evaluation_metrics = {}

    def prepare_model_data(self):
        predictor_matrix = self.dataset[self.predictors]
        outcome_series = self.dataset[self.outcome_var]
        return predictor_matrix, outcome_series

    def build_multiple_regression(self):
        print("构建多元线性回归模型...")

        X_features, y_target = self.prepare_model_data()
        X_with_intercept = sm.add_constant(X_features)

        self.mlr_fitted = sm.OLS(y_target, X_with_intercept).fit()
        predictions = self.mlr_fitted.predict(X_with_intercept)

        r_squared = self.mlr_fitted.rsquared
        adj_r_squared = self.mlr_fitted.rsquared_adj
```

```

mse_value = mean_squared_error(y_target, predictions)
mae_value = mean_absolute_error(y_target, predictions)

self.evaluation_metrics['MLR'] = {
    'R2': r_squared,
    'Adj_R2': adj_r_squared,
    'MSE': mse_value,
    'MAE': mae_value,
    'F_statistic': self.mlr_fitted.fvalue,
    'F_pvalue': self.mlr_fitted.f_pvalue
}

print(f"MLR 模型构建完成 - R2: {r_squared:.4f}")
return self.mlr_fitted

def build_generalized_additive_model(self):
    print("构建广义可加模型...")

    X_features, y_target = self.prepare_model_data()
    X_matrix = X_features.values
    y_array = y_target.values

    n_features = len(self.predictors)
    if n_features == 5:
        gam_formula = s(0) + s(1) + s(2) + s(3) + s(4)
    elif n_features == 6:
        gam_formula = s(0) + s(1) + s(2) + s(3) + s(4) + s(5)
    else:
        gam_formula = sum(s(i) for i in range(n_features))

    self.gam_fitted = LinearGAM(gam_formula).fit(X_matrix, y_array)
    gam_predictions = self.gam_fitted.predict(X_matrix)

    pseudo_r2 = self.gam_fitted.statistics_['pseudo_r2']['explained_deviance']
    mse_gam = mean_squared_error(y_array, gam_predictions)
    mae_gam = mean_absolute_error(y_array, gam_predictions)

    self.evaluation_metrics['GAM'] = {
        'Pseudo_R2': pseudo_r2,
        'MSE': mse_gam,
        'MAE': mae_gam
    }

    print(f"GAM 模型构建完成 - 伪 R2: {pseudo_r2:.4f}")

```

```
return self.gam_fitted
```

附录八

问题一：数据预处理代码

```
import pandas as pd
import numpy as np

def parse_gestational_week(week_input):
    if isinstance(week_input, (int, float)):
        return float(week_input)
    if isinstance(week_input, str):
        cleaned_input = week_input.strip()
        if 'w+' in cleaned_input:
            components = cleaned_input.split('w+')
            try:
                return int(components[0]) + int(components[1]) / 7.0
            except (ValueError, IndexError):
                return np.nan
        elif 'w' in cleaned_input:
            try:
                return float(cleaned_input.replace('w', ''))
            except ValueError:
                return np.nan
        else:
            try:
                return float(cleaned_input)
            except (ValueError, TypeError):
                return np.nan
    return np.nan

class DataProcessor:
    def __init__(self, excel_path='附件.xlsx', sheet_identifier='男胎检测数据'):
        self.data_source = excel_path
        self.target_sheet = sheet_identifier
        self.processed_data = None
        self.dependent_var = 'Y 染色体浓度'

    def load_raw_data(self):
        try:
            raw_dataframe = pd.read_excel(self.data_source, sheet_name=self.target_sheet)
            return raw_dataframe
        except FileNotFoundError:
```

```

        print(f'错误: 未找到文件 '{self.data_source}''')
        return None

    def preprocess_columns(self, dataframe):
        dataframe.columns = dataframe.columns.str.strip().str.replace('\n', '')
        dataframe['检测孕周'] = dataframe['检测孕周'].apply(parse_gestational_week)
        return dataframe

    def clean_missing_values(self, dataframe):
        critical_columns = [self.dependent_var, '检测孕周', '孕妇 BMI']
        cleaned_df = dataframe.dropna(subset=critical_columns)
        return cleaned_df

    def execute_preprocessing(self):
        print("开始数据预处理流程...")

        raw_df = self.load_raw_data()
        if raw_df is None:
            return None

        processed_df = self.preprocess_columns(raw_df)
        final_df = self.clean_missing_values(processed_df)

        self.processed_data = final_df
        print(f'数据预处理完成，有效样本数: {len(final_df)}')
        return final_df

    def get_numeric_columns(self):
        if self.processed_data is None:
            return []
        return self.processed_data.select_dtypes(include=np.number).columns.tolist()

    def export_processed_data(self, output_path='processed_data.csv'):
        if self.processed_data is not None:
            self.processed_data.to_csv(output_path, index=False, encoding='utf-8')

```

附录九

问题二：Y 染色体浓度与孕妇 BMI 散点图代码

```

import matplotlib.pyplot as plt
import matplotlib
import seaborn as sns
import pandas as pd

```

```

def configure_plot_style():
    matplotlib.rcParams['font.family'] = ['SimHei', 'Microsoft YaHei']
    matplotlib.rcParams['axes.unicode_minus'] = False
    matplotlib.rcParams['font.size'] = 10

def plot_bmi_vs_tmin_scatter(tmin_shuju):
    plt.figure(figsize=(12, 8))
    sns.scatterplot(data=tmin_shuju, x='孕妇 BMI', y='t_min', alpha=0.6, edgecolor=None)
    sns.regplot(data=tmin_shuju, x='孕妇 BMI', y='t_min', scatter=False, color='red',
line_kws={'linestyle':'--'})
    plt.title('孕妇 BMI 与 Y 染色体浓度达标孕周(t_min)关系散点图', fontsize=18)
    plt.xlabel('孕妇 BMI', fontsize=14)
    plt.ylabel('Y 染色体浓度首次达标孕周 (t_min)', fontsize=14)
    plt.grid(True, linestyle='--', alpha=0.6)
    plt.savefig('bmi_vs_tmin_scatter.png', dpi=300)
    plt.close()

```

附录十

问题三：回归树 BMI 分组代码

```

class JiaoTi_Youhua_Qi:
    def __init__(self, all_individuals_df, prob_model, n_groups, w, bounds):
        self.individuals_df = all_individuals_df.copy()
        self.prob_model = prob_model
        self.n_groups = n_groups
        self.w = w
        self.bounds = bounds
        print("正在为每个个体缓存其概率函数...")
        self.prob_funcs = self._huancun_gailv_hanshu()

    def _huancun_gailv_hanshu(self):
        prob_funcs = {}
        individual_features_df = self.individuals_df[self.prob_model.feature_names]

        for idx, row in tqdm(individual_features_df.iterrows(),
total=len(individual_features_df), desc="缓存概率函数"):
            prob_funcs[idx] = self._wei_geti_chuangjian_gailv_hanshu(idx, row)
        return prob_funcs

    def _wei_geti_chuangjian_gailv_hanshu(self, individual_id, features_row):
        def geti_gailv_hanshu(t, sigma_override=None):
            t_arr = np.array([t] if isinstance(t, (int, float)) else np.asarray(t)

```

```

X_pred = pd.DataFrame(np.tile(features_row.values, (len(t_arr), 1)),
                        columns=self.prob_model.feature_names)
X_pred[self.prob_model.feature_names[0]] = t_arr

y_hat = self.prob_model.yuce_nongdu(X_pred, individual_id)

current_sigma = sigma_override if sigma_override is not None else
self.prob_model.residual_std

from scipy.stats import norm
z_score = (self.prob_model.threshold - y_hat) / current_sigma
return 1 - norm.cdf(z_score)
return geti_gailv_hanshu

def _xunzhao_geti_zuiyou_shijian(self, prob_func, sigma_override=None):
    obj_func = lambda t: zong_mubiao_hanshu(t, prob_func, self.w,
sigma_override=sigma_override)
    result = minimize_scalar(obj_func, bounds=self.bounds, method='bounded')
    return result.x

def _chushihua_fenzu(self):
    if '孕妇 BMI' not in self.individuals_df.columns:
        self.individuals_df['孕妇 BMI'] = self.individuals_df['体重'] /
(self.individuals_df['身高'] / 100)**2

    kmeans = KMeans(n_clusters=self.n_groups, random_state=42, n_init=10)
    self.individuals_df['group'] = kmeans.fit_predict(self.individuals_df[['孕妇 BMI']])
    print(f'已基于 BMI 初始化 {self.n_groups} 个分组。")

def zhixing_youhua(self, max_iterations=10, tolerance=1e-3, sigma_override=None):
    self._chushihua_fenzu()
    group_timepoints = np.zeros(self.n_groups)

    for i in range(max_iterations):
        print(f"\n--- 迭代 {i+1}/{max_iterations} ---")

        print("步骤 A: 优化各组时间点...")
        new_group_timepoints = []
        for k in range(self.n_groups):
            group_member_indices = self.individuals_df[self.individuals_df['group'] ==
k].index

            if len(group_member_indices) == 0:
                new_group_timepoints.append(group_timepoints[k])

```

```

        continue

        individual_optimal_ts = [self._xunzhao_geti_zuiyou_shijian(self.prob_funcs[idx], sigma_override) for idx in group_member_indices]
        new_group_timepoints.append(np.median(individual_optimal_ts))

    new_group_timepoints = np.array(new_group_timepoints)

    print("步骤 B: 优化个体分组...")

    cost_matrix = np.zeros((len(self.individuals_df), self.n_groups))

    for k, t_k in enumerate(new_group_timepoints):
        probabilities = self.prob_model.piliang_gailv_jisuan(t_k, self.individuals_df, sigma_override)
        risk_at_tk = linchuang_chuangkou_fengxian(t_k)

        costs = self.w * risk_at_tk - (1 - self.w) * probabilities
        cost_matrix[:, k] = costs

    new_groups = np.argmin(cost_matrix, axis=1)

    change = np.linalg.norm(new_group_timepoints - group_timepoints)
    num_changed_assignments = np.sum(self.individuals_df['group'].values != new_groups)
    print(f"组时间点变化量: {change:.4f}, 组员变动数量: {num_changed_assignments}")

    group_timepoints = new_group_timepoints
    self.individuals_df['group'] = new_groups

    if change < tolerance and num_changed_assignments == 0:
        print("\n 算法已收敛! ")
        break

```

附录十一

问题二：敏感性分析代码

```

def zhixing_ming anxing_fenxi(unique_individuals, prob_model, config):
    print("\n--- 开始敏感性分析 ---")
    original_sigma = prob_model.residual_std
    results = []

```

```

for multiplier in config["sensitivity_sigma_multipliers"]:
    current_sigma = original_sigma * multiplier
    print(f"\n 正在为 sigma = {current_sigma:.4f} (乘数: {multiplier}) 运行分析...")

    optimizer = JiaoTi_Youhua_Qi(unique_individuals, prob_model,
                                config["n_groups"], config["optimizer_w"],
                                config["optimization_bounds"])

    final_groups_df, final_timepoints = optimizer.zhixing_youhua(sigma_override=current_sigma,
                                                                max_iterations=config["max_iterations"])

    for k in range(optimizer.n_groups):
        group_size = np.sum(final_groups_df['group'] == k)
        if group_size > 0:
            stable_group_label = f"T*≈{round(final_timepoints[k])} 周"
            results.append({
                "sigma": current_sigma,
                "stable_group": stable_group_label,
                "size": group_size,
                "timepoint": final_timepoints[k]
            })

```

附录十二

问题三：达标概率预测曲线代码

```

def shezhi_huitu_yangshi(config):
    sns.set_style("whitegrid")
    matplotlib.rcParams['font.family'] = config["plot_font"]
    matplotlib.rcParams['axes.unicode_minus'] = False

def huizhi_yangben_gailv_quxian(weeks_data, raw_probs, corrected_probs, sample_id):
    plt.figure(figsize=(12, 7))

    plt.plot(weeks_data, raw_probs, label='原始 GAM 预测概率 (可能波动)', linestyle='--',
             alpha=0.7)
    plt.plot(weeks_data, corrected_probs, label='PAVA 单调修正后概率', linewidth=2.5)

    plt.title(f'样本孕妇 {sample_id} 的 Y 染色体浓度达标概率曲线')
    plt.xlabel('孕周 (t)')
    plt.ylabel('达标概率 P(t)')
    plt.grid(True, linestyle='--')
    plt.ylim(0, 1)

```

```
plt.legend()
plt.savefig('sample_probability_curve_corrected.png')
plt.close()
```

附录十三

问题三：数据处理代码

```
import pandas as pd
import numpy as np
import warnings

warnings.filterwarnings("ignore", category=FutureWarning)

CONFIG = {
    "data_path": '附件_完整预处理.csv',
    "features": ['孕周_周数', '年龄', '身高', '体重'],
    "target": 'Y 染色体浓度',
    "individual_id_col": '孕妇代码',
    "probability_threshold": 0.04,
    "n_groups": 5,
    "optimizer_w": 0.6,
    "max_iterations": 10,
    "tolerance": 1e-3,
    "plot_font": ['SimHei', 'Microsoft YaHei'],
    "optimization_bounds": (10, 30),
    "sensitivity_sigma_multipliers": [0.8, 1.0, 1.2],
}

def jiazai_yuchuli_shuju(config):
    try:
        data = pd.read_csv(config["data_path"], encoding='utf-8-sig')
    except Exception as e:
        print(f'数据加载失败: {e}')
        return None, None

    male_fetus_data = data[data[config["target"]].notna()].copy()

    subset_cols = config["features"] + [config["target"], config["individual_id_col"]]
    male_fetus_data.dropna(subset=subset_cols, inplace=True)

    unique_individuals = male_fetus_data.drop_duplicates(subset=[config["individual_id_col"]]).set_index(config["individual_id_col"])
```

```

    print(f"数据准备完成。共 {len(male_fetus_data)} 条检测记录, {len(unique_individuals)}
    位独立个体。")
    return male_fetus_data, unique_individuals

def huoqu_yangben_geti_shuju(data, config, sample_id='A001'):
    if sample_id not in data[config["individual_id_col"]].values:
        sample_id = data[config["individual_id_col"]].iloc[0]

    sample_features = data[data[config["individual_id_col"]].values ==
sample_id][config["features"]].iloc[0]
    return sample_id, sample_features

def chuangjian_yingbianma_minganxing_shuju():
    final_results_data = [
        {'sigma': 0.0235, 'groups': [
            {'group_id': 2, 'size': 35, 'timepoint': 14.0},
            {'group_id': 3, 'size': 170, 'timepoint': 12.0},
            {'group_id': 4, 'size': 62, 'timepoint': 14.3}
        ]},
        {'sigma': 0.0294, 'groups': [
            {'group_id': 1, 'size': 185, 'timepoint': 12.0},
            {'group_id': 2, 'size': 34, 'timepoint': 14.1},
            {'group_id': 4, 'size': 48, 'timepoint': 14.2}
        ]},
        {'sigma': 0.0353, 'groups': [
            {'group_id': 3, 'size': 212, 'timepoint': 12.0},
            {'group_id': 4, 'size': 55, 'timepoint': 14.2}
        ]}
    ]

    sensitivity_df_data = []
    for res in final_results_data:
        sigma_val = res['sigma']
        for group in res['groups']:
            stable_group_label = f"T*≈ {round(group['timepoint'])}周"
            sensitivity_df_data.append({
                "sigma": sigma_val,
                "stable_group": stable_group_label,
                "size": group['size'],
                "timepoint": group['timepoint']
            })

    sensitivity_df = pd.DataFrame(sensitivity_df_data)

```

```

sensitivity_df_agg = sensitivity_df.groupby(['sigma', 'stable_group']).agg(
    total_size=('size', 'sum'),
    avg_timepoint=('timepoint', 'mean')
).reset_index()

return sensitivity_df_agg

def shengcheng_yangben_gailv_quxian_shuju(prob_model, sample_id, sample_features):
    weeks_fine = np.linspace(10, 25, 100)

    def huoqu_keqiehuan_gailv_hanshu(individual_id, individual_features, apply_pava=True):
        if prob_model.gam_model is None:
            raise RuntimeError("模型尚未训练。")

        def prob_func(t, threshold=0.04, sigma_override=None):
            if isinstance(t, (int, float)):
                t = np.array([t])

            X_pred = pd.DataFrame(np.tile(individual_features.values, (len(t), 1)),
                                   columns=prob_model.feature_names)
            X_pred[prob_model.feature_names[0]] = t

            y_hat = prob_model.yuce_nongdu(X_pred, individual_id)

            current_sigma = sigma_override if sigma_override is not None else
prob_model.residual_std

            from scipy.stats import norm
            z_score = (threshold - y_hat) / current_sigma
            probability = 1 - norm.cdf(z_score)

            if apply_pava:
                mono_probability = prob_model.isotonic_regressor.fit_transform(t, probability)
                return mono_probability
            else:
                return probability

        return prob_func

    raw_prob_func = huoqu_keqiehuan_gailv_hanshu(sample_id, sample_features,
                                                    apply_pava=False)
    corrected_prob_func = huoqu_keqiehuan_gailv_hanshu(sample_id, sample_features,
                                                         apply_pava=True)

```

```

raw_probs = raw_prob_func(weeks_fine)
corrected_probs = corrected_prob_func(weeks_fine)

return weeks_fine, raw_probs, corrected_probs

```

附录十四

问题四：分层模型训练代码

```

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, precision_recall_curve
from sklearn.impute import SimpleImputer
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline

def train_screening_model(X, y, features):
    print("\n--- 4. 训练第一层：高灵敏度筛查模型 ---")

    X_selected = X[features]

    X_train, X_test, y_train, y_test = train_test_split(
        X_selected, y, test_size=0.25, random_state=42, stratify=y
    )

    print(f"训练集大小: {X_train.shape[0]}, 测试集大小: {X_test.shape[0]}")
    print(f"训练集中异常样本比例: {y_train.mean():.2%}")
    print(f"测试集中异常样本比例: {y_test.mean():.2%}")

    pipeline = ImbPipeline([
        ('smote', SMOTE(random_state=42)),
        ('scaler', StandardScaler()),
        ('classifier', LogisticRegression(random_state=42, class_weight='balanced',
solver='liblinear', C=1.0))
    ])

    pipeline.fit(X_train, y_train)

```

```

y_pred_proba = pipeline.predict_proba(X_test)[: , 1]
precision, recall, thresholds = precision_recall_curve(y_test, y_pred_proba)

target_recall = 0.99
high_recall_indices = np.where(recall >= target_recall)[0]

if len(high_recall_indices) > 0:
    best_idx = high_recall_indices[np.argmax(precision[high_recall_indices])]
    optimal_threshold = thresholds[best_idx]
else:
    optimal_threshold = thresholds[np.argmin(recall)]

print(f"\n 为达到召回率 >={target_recall*100}%, 优化后的概率阈值为 :
{optimal_threshold:.4f}")

y_pred_optimal = (y_pred_proba >= optimal_threshold).astype(int)

print("\n 在测试集上的最终评估 (使用优化阈值):")
print(classification_report(y_test, y_pred_optimal))
print("混淆矩阵:")
print(confusion_matrix(y_test, y_pred_optimal))

return pipeline, optimal_threshold

def train_typing_model(X, y, features, task_name):
    print(f"\n--- 5. 训练第二层: 高精度分型模型 - 任务: {task_name} ---")

    X_selected = X[features]

    X_train, X_test, y_train, y_test = train_test_split(
        X_selected, y, test_size=0.25, random_state=42, stratify=y
    )

    print(f"训练集大小: {X_train.shape[0]}, 测试集大小: {X_test.shape[0]}")

    pipeline = ImbPipeline([
        ('smote', SMOTE(random_state=42, k_neighbors=2)),
        ('scaler', StandardScaler()),
        ('classifier', LogisticRegression(random_state=42, multi_class='multinomial'))
    ])

    param_grid = {
        'classifier__C': [0.1, 1, 10, 100],
        'classifier__penalty': ['l1', 'l2'],

```

```

        'classifier__solver': ['liblinear', 'saga']
    }

    grid_search = GridSearchCV(pipeline, param_grid, cv=3, scoring='f1_macro', n_jobs=-1)
    grid_search.fit(X_train, y_train)

    print(f"\n 最佳超参数: {grid_search.best_params_}")

    y_pred = grid_search.predict(X_test)

    print("\n 在测试集上的最终评估:")
    print(classification_report(y_test, y_pred, target_names=['T21', 'T18', 'T13']))
    print("混淆矩阵:")
    print(confusion_matrix(y_test, y_pred))

    return grid_search.best_estimator_

def train_advanced_screening_model(X_train, X_test, y_train, y_test, features, output_dir):
    print("\n--- 训练第一层: 高灵敏度筛查模型 ---")

    X_train_selected = X_train[features]
    X_test_selected = X_test[features]

    pipeline = ImbPipeline([
        ('imputer', SimpleImputer(strategy='median')),
        ('smote', SMOTE(random_state=42)),
        ('scaler', StandardScaler()),
        ('classifier', LogisticRegression(random_state=42, class_weight='balanced', C=1.0))
    ])
    pipeline.fit(X_train_selected, y_train)

    y_pred_proba = pipeline.predict_proba(X_test_selected)[:, 1]
    precision, recall, thresholds = precision_recall_curve(y_test, y_pred_proba)

    plt.figure(figsize=(10, 7))
    plt.plot(recall, precision, marker='.', label='精确率-召回率曲线')

    target_recall = 0.99
    high_recall_indices = np.where(recall >= target_recall)[0]

    if len(high_recall_indices) > 0:
        best_idx = high_recall_indices[np.argmax(precision[high_recall_indices])]
        optimal_threshold = thresholds[best_idx]
        optimal_recall = recall[best_idx]

```

```

    optimal_precision = precision[best_idx]

    plt.scatter(optimal_recall, optimal_precision, marker='o', color='red', s=100, zorder=5,
                label=f'最优点 (召回率={optimal_recall:.2f}, 精确率={optimal_precision:.2f})\n  阈值={optimal_threshold:.4f}')
    else:
        best_idx = np.argmax(recall)
        optimal_threshold = thresholds[best_idx-1] if best_idx > 0 else thresholds[0]

    plt.xlabel('召回率 (Recall)')
    plt.ylabel('精确率 (Precision)')
    plt.title('第一层筛查模型：精确率-召回率曲线与最优点选择')
    plt.legend()
    plt.grid(True)
    plt.savefig(f'{output_dir}/screen_model_pr_curve.png', dpi=300)
    plt.show()

    print(f'优化后的概率阈值为: {optimal_threshold:.4f}')
    return pipeline, optimal_threshold

def train_svm_typing_model(X_abnormal_train, y_abnormal_train_type, type_features):
    pipeline_svm = ImbPipeline([
        ('smote', SMOTE(random_state=42, k_neighbors=2)),
        ('scaler', StandardScaler()),
        ('classifier', SVC(random_state=42, class_weight='balanced', C=1, gamma='scale',
kernel='rbf'))
    ])
    pipeline_svm.fit(X_abnormal_train[type_features], y_abnormal_train_type)
    return pipeline_svm

```